

Introduction to Software Reverse Engineering

By Dr. Junjie Zhang

Learning Outcomes

- Explain what software reverse engineering is
- Distinguish static and dynamic analysis at a high level
- Explain what reverse engineers try to recover
- Explain why reverse engineering is reconstruction rather than simple inversion

What Reverse Engineering Is

Reverse engineering is the process of recovering useful information from software artifacts when source-level understanding is missing or incomplete.

Artifacts include:

- Binaries
- Libraries
- Obfuscated scripts
- Documents with embedded code

What Analysts Try to Recover

- Program behavior
- Important checks and logic
- Inputs and outputs
- Library and system interactions
- Data structures and control flow

Key idea:

- Reverse engineering is evidence-driven

Static vs Dynamic Analysis

Static:

- Analyze without execution
- Strings, symbols, disassembly, decompiler views

Dynamic:

- Analyze while executing
- Debugger traces, runtime behavior, environment effects

Real workflows often use both.

Reconstruction, Not Inversion

Compilation:

```
source -> binary
```

Reverse engineering:

```
binary -> layered reconstruction of meaning
```

Information is lost during compilation, so reverse engineering must infer.

Applications of Software Reverse Engineering

- Software understanding
- Interoperability
- Vulnerability research
- Malware investigation
- Digital forensics
- Legacy-system maintenance

Course Framing

This course is:

- Linux-first
- C/GCC/ELF centered
- Ghidra-supported
- Beginner-aware but technically serious
- Built around concepts and skills that also apply to Windows binaries, object files, and other software artifacts

Responsible Reverse Engineering Practice

By Dr. Junjie Zhang

Learning Outcomes

- Explain why reverse engineering goals affect safe practice
- Distinguish software understanding, malware-oriented analysis, and vulnerability-oriented analysis
- Describe safe handling habits for unknown binaries
- Explain why controlled environments matter in malware-oriented work
- Follow a basic CERT-based reporting path for vulnerability findings
- Explain why careful communication matters

Different Goals, Different Responsibilities

- Reverse engineering is not one single activity
- Different goals create different risks
- Different risks require different habits
- The goal of this module is practical judgment

Three Common Goals

1. Software understanding
2. Malware-oriented analysis
3. Vulnerability-oriented analysis

Each goal is legitimate.

Each goal also creates different responsibilities.

Core Questions Before Action

- What is my goal?
- What could go wrong if I do this carelessly?
- Am I using a safe environment?
- Do I need to run this binary at all?
- If I find something important, how should I communicate it?

Software Understanding

Common examples:

- Learning how a program works
- Understanding a legacy internal tool
- Inspecting an unknown binary before trusting it
- Comparing source-level ideas to binary behavior

Responsibilities In Software Understanding

- Curiosity is fine
- Start with static inspection when possible
- Avoid casual execution on a personal machine
- Keep notes on what you observe
- Separate facts from guesses

Malware-Oriented Analysis

Common examples:

- Studying suspicious binaries
- Investigating possible malicious behavior
- Looking for persistence, network activity, or payload behavior
- Supporting defensive response or forensics

Responsibilities In Malware-Oriented Analysis

- Assume the sample may be harmful
- Protect the host operating system
- Protect files, accounts, and network access
- Avoid casual sharing of samples
- Treat containment as part of the analysis

Controlled Environments Matter

For malware-oriented work, prefer:

- Virtual machines or disposable lab systems
- Snapshots you can roll back
- Limited file sharing with the host
- Restricted network access when needed
- Separate accounts and non-production systems

Vulnerability-Oriented Analysis

Common examples:

- Finding a security weakness
- Confirming risky program behavior
- Studying an attack path or unsafe design
- Preparing a responsible report for the owner

Reporting Paths

Default rule:

- Avoid public posting first
- Contact the vendor first when possible
- Use a coordinator if needed
- Use a bug bounty program only when the program covers that target

Vendor First

- Check the vendor's security page or `SECURITY.md`
- Look for `security.txt`, `security@`, or PSIRT contact
- Follow published reporting instructions
- Prepare a clear report with evidence and impact

Example:

- Node.js security page: `github.com/nodejs/node/security`

Coordinator (Example: CERT/CC)

- CERT Coordination Center is a vulnerability coordinator
- It helps when vendors are hard to reach
- It is useful when multiple vendors may be affected
- It focuses on coordination and harm reduction

Links:

- Guidance: kb.cert.org/vuls/guidance/
- Report a vulnerability: kb.cert.org/vuls/report/

What To Include In A Report

- Product name and version
- How you found the issue
- Clear steps or proof of concept
- Evidence and notes
- Impact explained carefully
- Uncertainty and limits

Bug Bounty Programs

- Bug bounty programs offer financial incentives
- The goal is ethical disclosure of real bugs or vulnerabilities
- Reports must focus on systems the program allows
- Programs define scope, rules, and reporting channels

Popular Bug Bounty Examples

- GitHub Bug Bounty

`bounty.github.com`

- Google Bug Hunters / VRP

`bughunters.google.com`

- Microsoft MSRC and bounty programs

`microsoft.com/msrc`

- HackerOne public bug bounty programs

`hackerone.com/bug-bounty-programs`

- Bugcrowd public bug bounty list

`bugcrowd.com/bug-bounty-list/` 17

Professional Practice

Good reverse engineering practice requires:

- Technical accuracy
- Restraint
- Risk awareness
- Honest reporting

Context of Software Reverse Engineering: Compilation and Reconstruction

By Dr. Junjie Zhang

Learning Outcomes

- Describe Compilation As A Translation Pipeline
- Explain Reverse Engineering As Reconstruction From Lower-Level Artifacts
- Identify What Information Is Lost During Compilation
- Explain Why Intermediate Representations Help Both Compilation And Reverse Engineering
- Explain Why Decompiled Code Differs From The Original Source Code

Compilation

source code -> intermediate representation -> assembly/object code -> executable binary

- Higher-level intent is translated into lower-level forms
- Source code is designed for human readability
- IR is a convenient level for analysis and optimization
- Assembly and binary forms are designed for processors, not human readers

Where Information Is Lost

- Comments are removed
- Original variable names are usually lost
- Many type details become unclear
- High-level structure is flattened into lower-level control flow
- Compiler optimizations may merge, reorder, or eliminate code patterns

Reverse Engineering

binary -> disassembly -> lifted intermediate representation -> decompiled approximation -> human interpretation

- We begin with the artifact we have, not the source we want
- Reverse engineering works upward from machine-level evidence
- The goal is understanding, not perfect recovery of the original program text
- Reverse engineering must infer what compilation no longer preserves

From Binary To Assembly: Challenges

- Machine code to assembly is usually a high-fidelity translation of the same low-level behavior
- But disassembly does not recover the original authored assembly file
- Labels, comments, and original symbol names may be missing
- Code boundaries and intended control-flow structure may not always be obvious

Example: Authored Assembly

Compiled / authored assembly:

```
; int add_and_check(int x, int y)
add_and_check:
    add    edi, esi           ; sum = x + y
    cmp    edi, 100
    jg     too_large
    mov    eax, edi
    ret
too_large:
    mov    eax, -1
    ret
```

- Meaningful names, comments, and the structure are helpful.

Example: Disassembled Output

Disassembled assembly:

```
FUN_401020:  
    add    edi, esi  
    cmp    edi, 0x64  
    jg     LAB_40102c  
    mov    eax, edi  
    ret  
LAB_40102c:  
    mov    eax, 0xffffffff  
    ret
```

- The instruction behavior is nearly the same. But names, comments, and higher-level intent are reduced⁸ or missing.

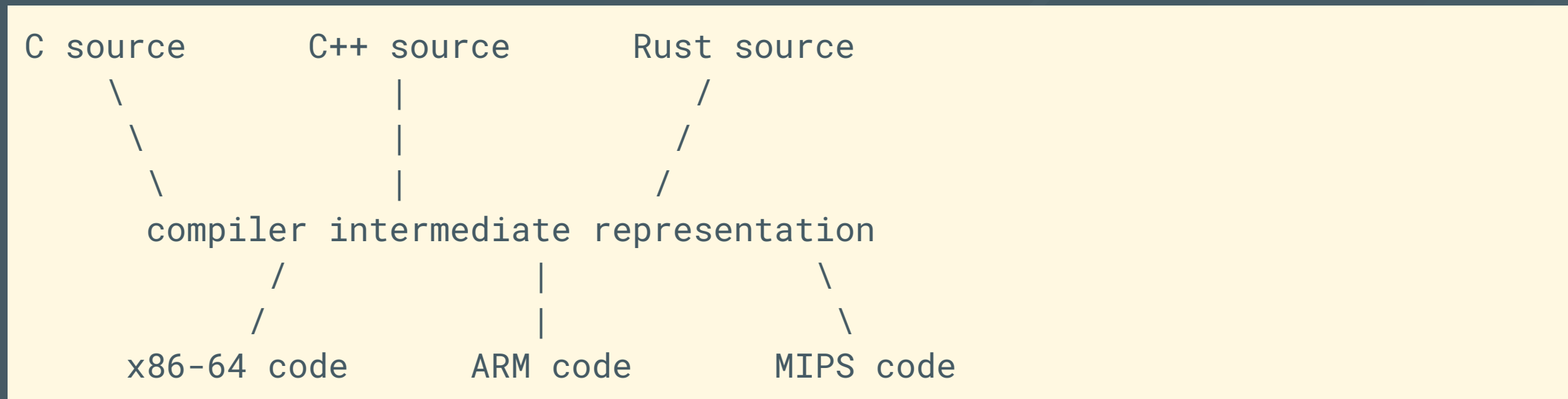
From Assembly To IR: Challenges

- Architecture-specific details must be abstracted into a more regular form
- The lifted IR is built for analysis, not necessarily to recover the compiler's original IR
- Some low-level behavior may only be represented approximately at the IR level

From IR To Decompiled Source: Challenges

- Lost names, comments, types, and abstractions do not automatically come back
- Program structure may be reconstructed differently from the original source
- Optimizations can hide intent and the original shape of the code
- Decompiled code is useful, but it is still an approximation rather than exact recovery

Why IR Matters In Compilation

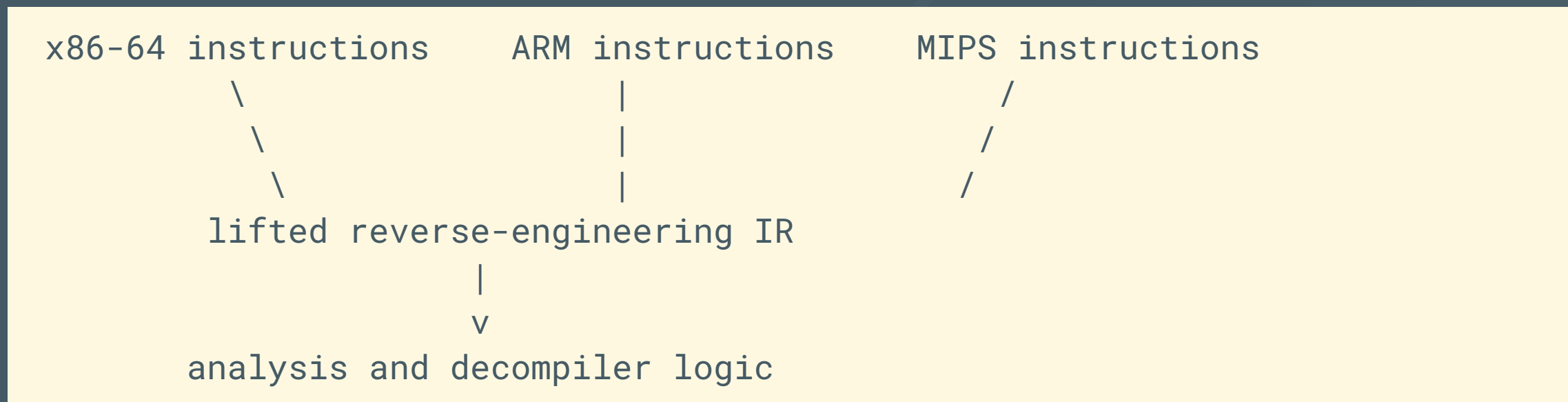


- Different source languages can be lowered into one common internal form
- That common form can then be translated into multiple target-specific outputs

Why IR Matters In Compilation

- IR helps abstract away source-language diversity
- Compilers analyze and transform programs more easily at the IR level
- Optimizations are often expressed over IR rather than raw source text
- IR creates a bridge between human-written code and machine-oriented code

Why IR Matters In Reverse Engineering



- One reverse-engineering IR can serve as a common analysis layer
- Different instruction sets can be lifted into that shared form

Why IR Matters In Reverse Engineering

- Reverse engineering IR abstracts away architecture diversity
- IR helps normalize architecture-specific instructions
- IR makes data flow, control flow, and operations easier to inspect
- The IR used for lifting is not necessarily the IR that the compiler used internally
- In Ghidra, p-code plays this role

Core Takeaway

- Compilation lowers meaning into executable form
- Reverse engineering raises meaning back from incomplete evidence
- Intermediate representations help bridge that gap
- Decompiled source is useful, but it is not the original source code

Environment Setup

By Dr. Junjie Zhang

Lecture Roadmap

1. Compilation Suite
2. Command-Line Tool Set
3. Integrated Tools
4. Validation Workflow

Module Focus

This module only focuses on the tools we will use early in the course.

- Compilation suite:
 - `build-essential`
 - 32-bit compilation support
- Command-line tool set:
 - `binutils`
- Integrated tools:
 - Ghidra

Recommended Virtual Machine Environment

1. **Preferred:** use Linux, Windows, or Mac as the host, install VirtualBox, then run Ubuntu or Kali in a virtual machine; install the reverse engineering course tools inside the virtual machine
2. **Windows alternative:** install Ghidra natively, then use Ubuntu in WSL for `build-essential` and `binutils`
3. **Linux/Mac alternative:** install the required tools directly in the native operating system

build-essential

- Platform: Debian / Ubuntu style Linux
- Purpose: Install the basic native build toolchain
- Typically includes: `gcc`, `g++`, `make`, and related development tools
- Course use: Compile small C examples and reproduce simple binaries for analysis

32-Bit Compilation Support

- Target scenario: build 32-bit binaries on a 64-bit Linux system
- Common Debian / Ubuntu style packages:
 - `gcc-multilib`
 - `libc6-dev-i386`
- Support `-m32` builds for comparison and compatibility exercises
- Students can use it to compare 32-bit and 64-bit compilation output, especially differences in generated assembly

binutils

- Platform: Linux
- Purpose: Provide core binary inspection and development utilities
- Important tools for this course:
 - `strings`
 - `readelf`
 - `nm`
 - `objdump`
- Course use: Inspect binaries before opening a larger GUI tool

What `binutils` Lets Us Do

- `strings`: pull out readable clues
- `readelf`: inspect ELF headers, sections, and segments
- `nm`: inspect symbols
- `objdump`: inspect disassembly and section details

Course use:

- These tools answer fast targeted questions from the terminal

Why Use Command-Line Tools First

- They are lightweight and fast
- They help students ask smaller, clearer questions
- They build binary format intuition before full GUI analysis
- They often answer simple questions without opening Ghidra

Ghidra

Ghidra is a free, open-source software reverse engineering framework created and maintained by the NSA.

It provides a full-featured environment for analyzing compiled programs, including disassembly, decompilation, graphing, and scripting across Windows, macOS, and Linux.

Ghidra

- Ghidra depends on Java
- Students need a suitable Java runtime or JDK installed first
- After Java is ready, Ghidra can be downloaded, extracted, and launched
- Course use: interactive disassembly, decompilation, and annotation

Why Ghidra Matters

- Ghidra is the main reverse engineering platform in this course
- It combines many analysis views in one environment
- It supports a richer workflow than command-line tools alone
- Detailed first-use workflow can be expanded in the next module

Other Popular RE Tools

- **radare2**: command-first and scriptable, available on multiple platforms
- **Binary Ninja**: polished commercial workflow, available on multiple platforms
- **IDA Pro**: mature commercial platform with strong industry presence
- **x64dbg**: popular Windows debugger, mainly used for Windows executables
- **011yDbg**: classic Windows debugger, mainly associated with older 32-bit Windows analysis

Why Ghidra For This Course

- Free to use
- Open source
- Strong disassembly and decompilation workflow
- Good cross-reference and navigation support
- Large user community and broad course accessibility

Ghidra Compared With Others

- Vs **radare2**: Ghidra is easier for beginners who need a strong GUI workflow
- Vs **Binary Ninja**: Ghidra avoids license cost barriers for students
- Vs **IDA Pro**: Ghidra is far more accessible for full-class adoption
- Across the course: one shared free platform makes demos and labs more consistent

Validation Workflow

1. Confirm the compiler toolchain works
2. Confirm `binutils` tools are available
3. Confirm optional 32-bit compilation works if installed
4. Confirm Java is installed
5. Confirm Ghidra launches successfully

Simple Validation Targets

- `gcc --version`
- `readelf --version`
- `objdump --version`
- `java --version`
- compile a small C example
- launch Ghidra successfully

Summary

- `build-essential` gives us the compilation baseline
- 32-bit support helps with later comparison exercises
- `binutils` gives us essential command-line inspection tools
- Java is required for Ghidra
- Ghidra is the main integrated analysis platform for the course

Sources

- Ghidra: <https://ghidra-sre.org>
- GNU Binutils: <https://www.gnu.org/software/binutils/>
- radare2: <https://rada.re/n/>
- Binary Ninja: <https://binary.ninja/>
- IDA Pro: <https://hex-rays.com/ida-pro/>
- Debian build-essential: <https://packages.debian.org/>
- Ubuntu packages: <https://packages.ubuntu.com/>

Environment Variables

By Dr. Junjie Zhang

Learning Outcomes

- Explain What An Environment Variable Is
- Explain Process Inheritance
- View, Set, Export, And Unset Variables
- Use One-Command Assignment
- Explain `PATH`, `getenv`, And `envp`

Lecture Roadmap

1. What Environment Variables Are
2. Setting Variables
3. Viewing Variables
4. `getenv` And `envp`
5. High-Value Examples Such As `PATH`
6. Inheritance And `export`
7. One-Command Inherited Assignment
8. Reverse Engineering Relevance

Why Environment Variables Matter

- They Affect Runtime Behavior
- They Influence Process Launch Context
- They Can Redirect Libraries Or Tools
- They Help Explain Why The Same Binary May Behave Differently

What An Environment Variable Is

- A Key-Value Pair In A Process Environment
- Typical Form: `VARIABLE=value`

Setting and Unsetting

Set:

```
MYVAR=hello
```

Unset:

```
unset MYVAR
```

Viewing Variables

List All:

```
env  
printenv
```

View One:

```
echo $PATH  
printenv HOME
```

getenv

- Access Environment Variables In C
- `getenv` Is The Simpler Name-Based Lookup

Example:

```
const char *v = getenv("APIKEY");  
printf("%s\n", v ? v : "not set");
```

envp

- Access Environment Variables In C
- `envp` Gives A Wider View Of The Process Environment

Example:

```
int main(int argc, char *argv[], char *envp[]) {  
    for (int i = 0; envp[i] != NULL; i++) {  
        if (strncmp(envp[i], "APIKEY=", 7) == 0)  
            printf("%s\n", envp[i]);  
    }  
    return 0;  
}
```

PATH As The First High-Value Example

- Controls Where The Shell Searches For Executables
- Helps Explain Why A Command May Or May Not Be Found
- Shows That Runtime Context Changes Behavior

Example:

```
echo $PATH
```

Other Popular Environment Variables

- HOME
- USER
- SHELL
- LANG

Inheritance And `export`

- Parent Processes Launch Child Processes
- Child Processes Usually Inherit Exported Environment Variables
- A Shell Variable Is Not Automatically Inherited
- `export` Marks A Variable For Child Process Inheritance

Example:

```
MYVAR=hello  
export MYVAR
```


One-Command Inherited Assignment

```
MYVAR=123 python3 -c 'import os; print(os.getenv("MYVAR"))'
```

- The Variable Is Passed To That Command Automatically
- Useful For Testing Child Process Inheritance Quickly
- Does Not Permanently Change The Current Shell

Other Relevant Variables

- LD_PRELOAD
- LD_LIBRARY_PATH
- PYTHONPATH

Important Note:

- Environment Variables Can Redirect Behavior Without Changing The Binary
- These Variables Are Often Unset By Default
- When Set, They Can Still Strongly Affect Runtime Behavior

Reverse Engineering Relevance

- Library Loading Can Change
- Instrumentation Behavior Can Change
- Runtime Experiments Depend On Launch Context
- Analysts Should Not Assume The Binary Alone Explains Everything

Summary

- Environment Variables Are Part Of Process Context
- Inheritance Explains Why `export` Matters
- One-Command Assignment Is Precise And Useful
- `PATH`, `getenv`, And `envp` Are Strong Early Examples

C Program Organization

By Dr. Junjie Zhang

Learning Outcomes

- Explain The Difference Between A Header File And A Source File
- Explain Interface Versus Implementation
- Explain Why Include Guards Matter
- Distinguish Declarations From Definitions
- Build And Run A Small Multi-File C Program

Lecture Roadmap

1. Why Multi-File Organization Exists
2. Declarations Versus Definitions
3. Interface Versus Implementation
4. Typical Project Layout
5. A Complete Small Example
6. What Goes In A Header
7. What Goes In A Source File
8. Include Guards

Why This Matters

- Real C Programs Rarely Stay In One File
- Developers Want Modularity And Reuse
- Independent Compilation Becomes Possible
- Reverse Engineers Later See The Results, Not The Original Layout
- Ghidra's Data Type Manager uses header file

Declaration Vs Definition

- Declaration
 - Tells The Compiler That Something Exists
- Definition
 - Provides The Actual Implementation Or Storage

Simple Rule:

- Many Uses May Depend On One Declaration
- The Real Thing Must Still Be Defined Somewhere

Declarations Versus Definitions

Declaration:

```
int threshold_is_high(const config_t *cfg);
```

Definition:

```
int threshold_is_high(const config_t *cfg) {  
    return cfg->threshold >= max;  
}
```

Interface Versus Implementation

```
Header (.h)  -> interface / contract  
Source (.c)  -> implementation
```

- Headers Describe What Exists
 - Declaration
- Source Files Show How It Works
 - Definition

Typical Layout

```
project/  
├── include/  
│   └── config_reader.h  
├── src/  
│   ├── config_reader.c  
│   └── main.c  
└── Makefile
```

- `include/` Holds Shared Interfaces
- `src/` Holds Implementations

A Complete Small Example

- `config_reader.h`
- `config_reader.c`
- `main.c`

Goal:

- Show The Interface
- Show The Implementation
- Show How `main.c` Uses The Interface

Example Header: `config_reader.h`

```
#ifndef CONFIG_READER_H
#define CONFIG_READER_H

typedef struct {
    const char *name;
    int threshold;
} config_t;

extern int max;

void print_config(const config_t *cfg);
int threshold_is_high(const config_t *cfg);

#endif
```

Example Source: `config_reader.c`

```
#include "config_reader.h"
#include <stdio.h>

int max = 10;

void print_config(const config_t *cfg) {
    printf("name=%s threshold=%d\n", cfg->name, cfg->threshold);
}

int threshold_is_high(const config_t *cfg) {
    return cfg->threshold >= max;
}
```

Example Entry Point: `main.c`

```
#include "config_reader.h"
#include <stdio.h>

int main(void) {
    config_t demo = {"demo-mode", max + 2};
    print_config(&demo);
    printf("max=%d high=%s\n",
          max,
          threshold_is_high(&demo) ? "yes" : "no");
    return 0;
}
```


What Goes In A Header

- Function Declarations
- Type Definitions
- Named Constants And Globals When Declared Carefully
- Later: `extern` Declarations

Avoid:

- Ordinary Function Bodies
- Global Variable Definitions
- Private Implementation Detail

Include Guards

```
#ifndef CONFIG_READER_H
#define CONFIG_READER_H

/* header contents */

#endif
```

- Headers May Be Included More Than Once
- Guards Prevent Redefinition Problems

What Goes In A Source File

- Function Definitions
- Private Helper Logic
- Implementation-Specific Includes

Good Habit:

```
#include "config_reader.h"
```

Include Your Own Header First.

How Multiple Files Cooperate

- `main.c` Includes The Header And Calls Functions
- `config_reader.c` Includes The Header And Defines Functions
- The Header Acts As A Shared Contract

Build View

One-Command Build:

```
gcc -Iinclude src/main.c src/config_reader.c -o config_demo
```

Separate Compilation Preview:

```
gcc -Iinclude -c src/main.c -o main.o  
gcc -Iinclude -c src/config_reader.c -o config_reader.o  
gcc main.o config_reader.o -o config_demo
```

Common Mistakes

- Missing Include Guards
- Putting Function Bodies In Ordinary Headers
- Defining Globals In Headers
- Forgetting The Matching Header In A `.c` File

Reverse Engineering Connection

- Source Layout Does Not Survive Directly In The Binary
- Some Symbol Clues May Survive
- Interface Ideas Often Need To Be Reconstructed
- Later Modules Build On This Source-Side Foundation

extern, **static**, **const**, and **Global Variables**

By [Dr. Junjie Zhang](#)

Learning Outcomes

- Explain What A Global Variable Is
- Distinguish Declaration From Definition For Variables
- Explain How `extern` Shares A Variable Across Files
- Explain File-Scope And Function-Scope `static`
- Explain Practical Uses Of `const`

Lecture Roadmap

1. Global Variables
2. `extern`
3. `static` At File Scope
4. `static` Inside A Function
5. `const`
6. Useful Combinations
7. Reverse Engineering Connection

Three Different Questions

- Where Does The Data Live?
- Who Can See It?
- Who Can Modify It?

These Questions Are Related, But They Are Not The Same.

One Running Example

Files:

- `counter.h`
- `helper.h`
- `counter.c`
- `helper.c`
- `main.c`

Global Variables

Example:

```
int shared_counter = 0;
```

- Lifetime = Whole Program
- Definition Lives In `counter.c`
- Other Files May Use It Through A Declaration

Declarations Versus Definitions

Declaration:

```
extern int shared_counter;
```

Definition:

```
int shared_counter = 0;
```

extern

```
/* counter.h */  
extern int shared_counter;
```

```
/* counter.c */  
int shared_counter = 0;
```

- Declaration Only
- No Storage Created Here
- Real Definition Must Exist Elsewhere

`static` At File Scope

```
/* counter.c */  
static int file_private_total = 10;
```

- Lifetime = Whole Program
- Visibility = Current `.c` File Only
- Another File Cannot Use `extern` To Reach It

static Inside A Function

```
/* counter.c */  
void call_counter_demo(void) {  
    static int counter_demo_calls = 0;  
    counter_demo_calls++;  
}
```

- Value Persists Across Calls
- Name Remains Local To The Function

const

```
/* counter.c */  
const int MAX_LIMIT = 32;
```

- Controls Mutability Through That Name
- Does Not Remove The Value From The Binary

Example Snapshot

```
/* counter.h */  
extern int shared_counter;  
extern const int MAX_LIMIT;  
void call_counter_demo(void);
```

```
/* counter.c */  
int shared_counter = 0;  
static int file_private_total = 10;  
const int MAX_LIMIT = 32;
```

```
/* helper.c */  
shared_counter += 1;  
printf("helper_report: shared_counter=%d, MAX_LIMIT=%d\n", shared_counter, MAX_LIMIT);
```

Useful Combinations

- `extern const int MAX_USERS;`
 - Shared Read-Only Value
- `static const int TABLE_SIZE = 256;`
 - File-Private Constant
- `static int request_count = 0;`
 - File-Private Global State

Reverse Engineering Connection

- `extern` Declarations Disappear After Compilation
- Definitions Become Real Storage
- These Choices Help Determine Which ELF Section Stores The Data
- File-Scope `static` Changes Symbol Visibility
- `const` Often Maps To Read-Only Regions

Compilation Pipeline Using `gcc`

By Dr. Junjie Zhang

Learning Outcomes

- Describe The Major Compilation Stages
- Follow The Artifact Chain `.i -> .s -> .o -> executable`
- Explain Preprocessing, Compilation, Assembly, And Linking
- Connect The Pipeline To Later Binary Artifacts

Lecture Roadmap

1. Big Picture
2. Running Example
3. Preprocessing
4. Compilation To Assembly
5. Assembly To Object Files
6. Linking
7. One Command Vs Full Pipeline
8. Reverse Engineering Connection

Big Picture

main.c, counter.c, helper.c

- > preprocessing
- > .i files
- > compilation
- > .s files
- > assembly
- > .o files
- > linking
- > executable

Running Example

Files:

- `counter.h`
- `helper.h`
- `main.c`
- `counter.c`
- `helper.c`

Preprocessing

Example:

```
gcc -E -Iinclude src/main.c -o main.i
```

- Expands `#include`
- Expands Macros
- Produces Preprocessed Source

Compilation To Assembly

Example:

```
gcc -S main.i -o main.s
```

Intel Syntax Option:

```
gcc -S -masm=intel main.i -o main_intel.s
```

- Uses Preprocessed Output
- Produces Assembly Text
- Makes Lower-Level Structure Visible

Assembly To Object Files

Example:

```
as main.s -o main.o
```

- Produces Machine-Code Object Files
- Not A Final Executable
- May Still Contain Unresolved References

Linking

Example:

```
gcc main.o counter.o helper.o -o state_demo
```

- Resolves Cross-File References
- Combines Object Files
- Produces The Final Executable

Full Pipeline Example

```
gcc -E -Iinclude src/main.c -o main.i  
gcc -S main.i -o main.s  
as main.s -o main.o
```

```
gcc -E -Iinclude src/counter.c -o counter.i  
gcc -S counter.i -o counter.s  
as counter.s -o counter.o
```

```
gcc -E -Iinclude src/helper.c -o helper.i  
gcc -S helper.i -o helper.s  
as helper.s -o helper.o  
gcc main.o counter.o helper.o -o state_demo
```

One Command Vs Full Pipeline

One Command:

```
gcc -Iinclude src/main.c src/counter.c src/helper.c -o state_demo
```

Full Pipeline:

- Better For Teaching
- Better For Seeing Artifacts
- Better For Separating Build Stages

Include Paths

Example:

```
gcc -E -Iinclude src/main.c -o main.i
```

Meaning:

- `-Iinclude` Adds A Header Search Path

Common Misunderstandings

- Headers Do Not Survive As Headers In The Final Binary
- `.o` Files Are Not Final Executables
- `gcc` Can Drive Multiple Stages, But The Stages Still Exist
- Linking Is Distinct From Compilation

Reverse Engineering Connection

- `.s` Files Preview Lower-Level Program Structure
- `.o` Files Already Contain Symbols And Relocations
- Linking Choices Affect What Later Tools Show
- This Module Bridges C Source To ELF Analysis

Static Libraries and Shared Libraries

By Dr. Junjie Zhang

Learning Outcomes

- Explain The Difference Between Headers And Libraries
- Explain Static Versus Shared Library Usage
- Use `-l` And `-L` In A Working Example
- Inspect Dependencies With `ldd` And Symbols With `nm`

Lecture Roadmap

1. What A Library Is
2. Static Libraries
3. Shared Libraries
4. Static Linking Vs Dynamic Linking
5. Using External Libraries With **-1**
6. Headers Vs Libraries
7. Useful Tools
8. Reverse Engineering Connection

What A Library Is

- Compiled Code Packaged For Reuse
 - Static Libraries Usually End In `.a`
 - Shared Libraries Usually End In `.so`

`.a` Vs `.so`

Static Library: `.a`

- Code Is Copied Into The Executable At Link Time
- Often Produces A Larger Executable
- Reduces Runtime Dependency On That Library File

Shared Library: `.so`

- Code Stays Outside The Executable
- Often Produces A Smaller Executable
- Requires The Library To Be Found At Runtime

Working Example: **libm**

Source:

```
#include <stdio.h>
#include <math.h>

int main(void) {
    printf("%0.1f\n", sqrt(16.0));
    return 0;
}
```

Link:

```
gcc main.c -lm -o demo
```

What `-l` Means

`-lm`

Means:

- Link Against `libm.so` Or `libm.a`
- General Rule: `-lfoo` Means `libfoo`
- Common Default Locations Include `/lib/x86_64-linux-gnu`, `/usr/lib/x86_64-linux-gnu`, `/lib`, And `/usr/lib`
- In This Demo, `libm.so` Is A Symlink To `libm.so.6`

Checking Default Library Paths

Show Default Search Paths:

```
gcc -print-search-dirs
```

- Useful For Seeing System Default Library Paths
- If The Library Is Already In One Of Those Default Paths, You Usually Do Not Need `-L`
- Use `-L` Mainly For Non-Standard Locations Such As A Project Folder Or `/usr/local/lib`

Why No Extra Flag For `printf`?

- `printf` Comes From The C Standard Library, `libc`
- In A Normal C Build, `gcc` Links `libc` Automatically
- So You Usually Do Not Need To Write `-lc`

By Contrast:

- `sqrt` Comes From The Math Library, `libm`
- `libm` Is Usually Not Linked Automatically
- So We Typically Write `-lm`

So We Usually Write:

Library Order Matters

Correct:

```
gcc main.c -lm
```

Incorrect:

```
gcc -lm main.c
```

- Libraries Usually Appear After Source Or Object Files

Library Search Paths: `-L`

If A Library Is Not In A Standard Location:

```
gcc main.c -Llocal_lib -lm -o demo
```

- `-L` Adds A Library Search Path
- The Linker Searches Those Paths Before Default System Locations
- In Our Demo, `local_lib/` Contains The Copied `libm.so` Link Name

Shared Library Use (Default)

```
gcc main.c -lm -o demo
```

- Usually Uses `libm.so`
- Smaller Executable
- Library Loaded At Runtime

Check:

```
ldd demo
```

Local Copy Demo

Demo Layout:

```
demos/  
  main.c  
  local_lib/  
    libm.so -> libm.so.6  
    libm.so.6
```

Prepare The Local Copy:

```
cp /lib/x86_64-linux-gnu/libm.so.6 local_lib/  
ln -sf libm.so.6 local_lib/libm.so
```


Runtime Search Path

Two Common Runtime Solutions:

1. Embed A Runtime Search Path

```
gcc main.c -Llocal_lib -lm \  
-Wl,-rpath,'$ORIGIN/local_lib' -o hello_with_rpath
```

2. Provide The Path When Running

```
LD_LIBRARY_PATH=./local_lib ./hello_without_rpath
```

Runtime Search Path

Important Distinction:

- `-Llocal_lib` Helps The Linker Find The Library During Build
- `LD_LIBRARY_PATH` And `rpath` Help The Loader Find The Library During Execution

Implications for Reverse Engineering:

- `rpath` or `runpath`: path info is embedded in the ELF binary
- `LD_LIBRARY_PATH`: path info is external to the binary

Static Library Use

```
gcc main.c -static -lm -o demo
```

- Uses `libm.a`
- Larger Executable
- No Runtime Dependency On `libm.so` For That Part

Check:

```
ldd demo
```

Headers Vs Libraries

Headers:

- Tell The Compiler What Exists

Libraries:

- Provide The Compiled Implementation

Example Pair:

- `#include <math.h>`
- `gcc main.c -Llocal_lib -lm -o demo`

Common Linker Errors

Cannot Find Library:

```
cannot find -lfoo
```

Often Means:

- Missing `-L` Path
- Library Not Installed

Useful Tools

`ldd`:

- What Shared Libraries Does This Executable Depend On?

Reverse Engineering Connection

- Imports Reveal External Dependencies
- Shared Libraries Move Some Behavior Outside The Main Executable
- Static Linking Can Put More Implementation Inside The Binary
- Linker Choices Change What An Analyst Sees
 - `rpath` or `runpath`: path info is embedded in the ELF binary
 - `LD_LIBRARY_PATH`: path info is external to the binary

Checking Default Header Paths

For Header Include Paths, Use:

```
gcc -E -x c - -v < /dev/null
```

- This Shows The Compiler's Default Header Search Paths
- Use This To Understand When `-I...` Is Needed
- `gcc -print-search-dirs` Is For Library And Tool Search Paths, Not Header Include Paths

Compiler Options and Optimization

By Dr. Junjie Zhang

Learning Outcomes

- Compare Debug And Non-Debug Builds
- Explain Stripped Versus Unstripped Binaries
- Explain Why Optimization Changes Source-To-Binary Similarity

Lecture Roadmap

1. Debug Vs Non-Debug
2. Optimization Levels
3. Debug Info And Recovery
4. Stripped Vs Unstripped
5. Packers
6. Typical Analysis Targets
7. Reverse Engineering Connection

Why This Matters

- The Same Source Can Produce Different Analysis Experiences
- Build Choices Change Names, Structure, And Clues
- Reverse Engineering Difficulty Is Often A Build Question Too

Debug Vs Non-Debug

Debug-Oriented Example:

```
gcc -g demo.c -o demo_debug
```

Non-Debug Example:

```
gcc demo.c -o demo_nodebug
```

- `-g` Adds Debug Information
- Non-Debug Builds May Omit Helpful Source-Level Clues

Optimization

- Optimization Can Reshape Code Structure In The Binary
- Common Effects May Include:
 - Dead Code Elimination
 - Inlining
 - Loop Transformations

Optimization Levels I

- `-O0`
 - Easier Source-To-Binary Mapping
- `-Og`
 - Debug-Friendly Optimization
- `-O1`
 - Basic Simplification And Dead Code Removal
- `-O2`
 - Strong General-Purpose Optimization

Optimization Levels II

- `-O3`
 - More Aggressive Optimization
- `-Os`
 - Optimize For Smaller Code Size
- `-Oz`
 - Even Smaller Code When Supported

Debug Info And Recovery

- `-g -O0` Is Often The Best Case For Source-Like Recovery
- Ghidra May Recover Much More Accurate:
 - Function Names
 - Variable Names
 - Type Information
 - Source-Level Structure
- But It Still Does Not Guarantee Perfect Recovery Of Every Type, Variable, Or Source Construct

Practical Comparison

- `-g -O0`
 - Best Case For Source-Like Recovery
- `-g` With Optimization
 - More Debug Info Exists, But Mapping Becomes Less Clean
- No `-g`
 - Far Fewer Source-Level Clues

Stripped Vs Unstripped

Example:

```
gcc demo.c -o demo_unstripped  
strip demo_unstripped
```

- Stripping Removes Helpful Symbol Information
- It Does Not Remove The Program's Behavior
- We Will Inspect The Binary-Level Differences In More Detail Later

Symbol Visibility

Compare:

```
nm demo_debug  
nm demo_stripped
```

- Debug-Oriented Builds Preserve More Clues
- Stripped Builds Remove Many Names

Packers: **UPX** Example

Example:

```
upx demo
```

- A Packer Rewrites The Executable To Make It Smaller Or Harder To Inspect Directly
- **UPX** Is A Common Linux Example
- Packing Is Different From Stripping
- The Real Code May Only Become Fully Visible After Unpacking Or Runtime Analysis

Typical Analysis Targets

- Distributed Binaries Under Analysis Are Often:
 - Built Without `-g`
 - Optimized
 - Stripped
- Benign Binaries Are Less Likely To Be Packed
 - Legitimate Software May Still Use Packing, Compression, Or Other Protection Techniques
- Malicious Binaries Are More Likely To Be Packed Or Otherwise Obfuscated

Common Misunderstandings

- A Source Function Does Not Always Survive As A Clear Binary Function
- Stripped Does Not Mean Encrypted
- Packed Does Not Mean The Program Logic Is Gone
- Optimization Changes Analysis Difficulty, Not Only Performance

Reverse Engineering Connection

- Build Choices Affect Available Names
- Build Choices Affect Function Boundaries
- Build Choices Affect Source-Level Similarity
- Packing Can Further Hide Structure Until Unpacking Or Execution
- These Questions Should Be Asked Before Opening Ghidra

Executable And Linkable Format (ELF) Fundamentals

By Dr. Junjie Zhang

Learning Outcomes

- Explain Why ELF Matters In Linux Reverse Engineering
- Identify An ELF Binary With `file`
- Explain The Difference Between Raw Bytes And Structured Format
- Map Source-Level Constructs Into Common ELF Sections
- Use `readelf` And Ghidra To Inspect ELF Metadata

Lecture Roadmap

1. One Running Example
2. ELF As Structure
3. ELF Layout Picture

Lecture Roadmap 2

- 7. Sections Vs Segments
- 8. Common ELF Sections
- 9. `readelf` Command Walkthrough
- 10. Source-To-Section Mapping
- 11. Debug, Non-Debug, And Stripped Comparison
- 12. Ghidra As Cross-Check

Common Executable Formats

- ELF: Executable and Linkable Format
 - The Standard Binary Format Used By Most Linux And Many Unix-Like Systems
- PE: Portable Executable
 - The Main Executable Format Used By Windows For `.exe` And `.dll` Files
- Mach-O: Mach Object
 - The Native Executable Format Used By macOS And Other Apple Platforms

One Running Example

This Module Uses One Small C Program With:

- Two Separately Compiled Source Files
- One Shared Header File
- An Initialized Global
- An Uninitialized Global
- Cross-Object References To Globals And One Function
- One Imported Function: `printf`

Compile Three Variants

```
gcc -g -Iinclude -c src/main.c -o main_debug.o  
gcc -g -Iinclude -c src/report.c -o report_debug.o  
gcc main_debug.o report_debug.o -o elf_demo_debug
```

First Contact: Raw Bytes

```
hexdump -C elf_demo_debug | head
```

- At First Glance, It Is Only Bytes
- Reverse Engineering Needs Structure, Not Just Hex

Second Contact: A File

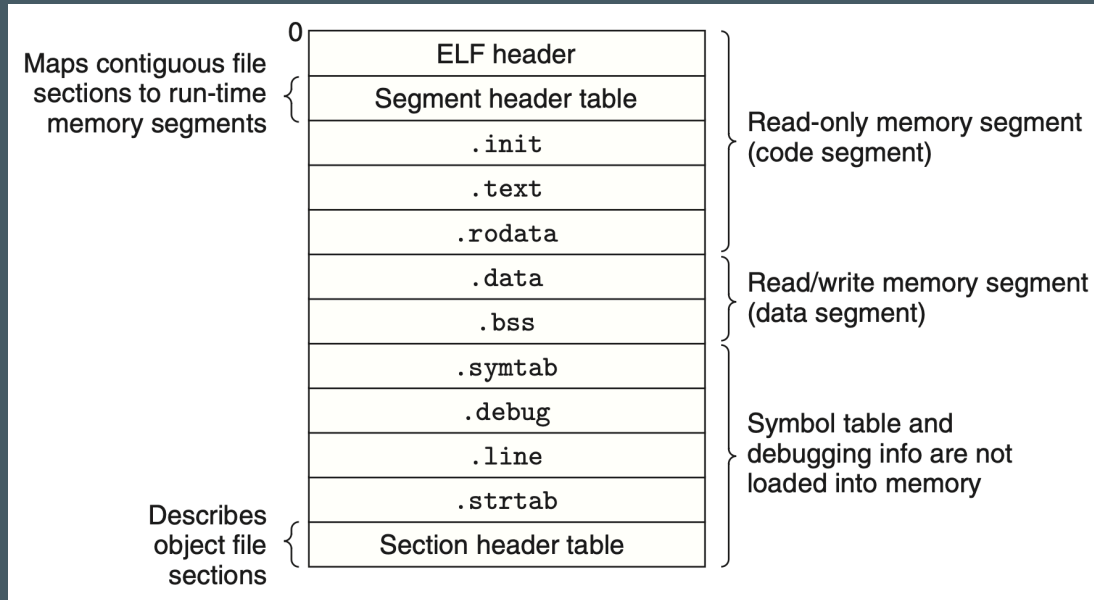
```
file elf_demo_debug
```

- ELF Classification
- Architecture And Bitness
- Dynamic Linking Clues
- Debug-Info Clues

ELF As Structure

- ELF Organizes Bytes Into Meaningful Parts
- The File Has Global Metadata
- The File Has Code And Data Regions
- The File Has Symbol And Import Information

ELF Layout Picture



Source:

- COMS 4995 Advanced Systems Programming
- <https://cs4157.github.io/www/2024-1/lect/15-elf-intro.html>

ELF Header

- Top-Level Metadata About The File
- Class, Machine, File Type
- Entry Point Or Starting Address
- Entry Point Is Not The Same As `main`

Segment Headers

- Describe How The OS Should Load Parts Of The File Into Memory
- Define Loadable Regions Such As Code, Data, And Permissions
- Belong To The Loader-Or Runtime-Oriented View Of ELF

Sections Vs Segments

Sections:

- Good For Linking And Reverse Engineering
- Organize Code, Data, Symbols, Strings, And Debug Info

Segments:

- Good For Program Loading
- Organize What The OS Maps Into Memory
- One Segment May Cover Multiple Sections

Two ELF Views

- Linker Or Section View
 - Merges Like Sections Across Object Files
- Loader Or Segment View
 - Groups Regions For Runtime Loading
- Not Every Section Is Loaded Into Memory
 - Example: Debug Information

Common ELF Sections

- `.text`
- `.rodata`
- `.data`
- `.bss`
- `.symtab` / `.strtab`
- `.dynsym` / `.dynstr`
- `.plt` / `.got`
- `.rela.*` / `.rel.*`
- `.debug_*`

Section Meanings: Code And Read-Only Data

- `.text`
 - Machine Code
 - `main`, `show_report`, `next_count`
- `.rodata`
 - Read-Only Strings And Constants
 - `banner`, `status_text`, `printf` Format String

Section Meanings: Writable Storage

- `.data`
 - Initialized Writable Globals
 - `initialized_count = 7`
- `.bss`
 - Uninitialized Global Or Static Storage
 - `pending_count`

Section Meanings: Symbol Tables

- `.symtab` / `.strtab`
 - Ordinary Symbol Names And Related Strings
 - Names Such As `main`, `show_report`, `next_count`, `initialized_count`
- `.dynsym` / `.dynstr`
 - Dynamic-Linking Symbol Names
 - Imported Names Such As `printf`

Section Meanings: Imports And Relocations

- `.plt` / `.got`
 - Dynamic-Call Resolution Support
 - Runtime Call Machinery For Imported Functions Such As `printf`
- `.rela.*`
 - Relocation Records
 - Linking And Loading Fixups For References Such As `show_report` And `printf`

Section Meanings: Debug Metadata

- `.debug_*`
 - Extra Debug Metadata From `-g`
 - Source-Line, Function, And Variable Help For `main.c` And `report.c`

readelf -h

```
readelf -h elf_demo_debug
```

- ELF Header
- Architecture, Bitness, File Type
- Entry Point
- Tells Us This Is An ELF Executable, Not Just Bytes

readelf -S

```
readelf -S elf_demo_debug
```

- Section Table
- Helps Map `.text`, `.data`, `.bss`, `.rodata`
- Shows Debug Sections In The `-g` Build

readelf -l

```
readelf -l elf_demo_debug
```

- Program Headers
- Shows Loadable Segments
- Connects File Layout To Runtime Memory Layout

readelf -s

```
readelf -s elf_demo_debug
```

- Symbol Information
- Function Names
- Global Variable Names
- Imported Function Names

readelf -r

```
readelf -r elf_demo_debug
```

- Relocation Information
- Imported-Function Resolution Clues
- Dynamic-Linking Evidence For `printf`

What Can We Derive From The Binary?

- Functions
 - `main`, `show_report`, `next_count`
- Globals
 - `initialized_count`, `pending_count`
- Imported Functions
 - `printf`

More Clues From The Binary

- Section Placement Clues
 - `.text`, `.data`, `.bss`, `.rodata`
- Debug Metadata
 - Extra Reconstruction Help In The `-g` Build

Debug Vs Non-Debug Vs Stripped

- Debug Build
 - Extra `.debug_*` Sections
 - More Source-Level Help
- Non-Debug Build
 - No Debug Sections
 - Many Function And Global Names May Still Survive
- Stripped Build
 - Many Names Disappear
 - Imports, Sections, And Core Layout Still Remain

Ghidra As The Integrating View

- Sections
- Segments
- Symbols
- Imports
- Strings
- Functions
- Memory Layout

It Makes ELF-Backed Structure Easier To Navigate.

Reverse Engineering Connection

- ELF Turns Bytes Into Interpretable Structure
- Build Choices Change What Survives
- Imported Functions And Sections Guide Analysis

Assembly Basics

By Dr. Junjie Zhang

Learning Outcomes

- Explain Why Reverse Engineers Read Assembly
- Explain Why This Module Focuses On x86-64 And Intel Syntax
- Recognize Common Registers, Memory References, And Operands
- Interpret `mov`, `lea`, `cmp`, `test`, `call`, And `ret`
- Recognize Common C-Like Patterns In Disassembly

Lecture Roadmap

1. Framing
2. One Running Demo
3. Computer Model
4. Registers, Memory, And Syntax
5. Core Instructions
6. Flags, Comparisons, And Branches

Lecture Roadmap 2

- 7. Calls, Returns, And Stack Basics
- 8. Recognizing Source-Level Patterns
- 9. One Combined Example
- 10. Optimization Caveat And Wrap-Up

Why Are We Learning Assembly?

- To Recognize What A Binary Is Doing
- To Connect Low-Level Instructions To High-Level Logic
- To Explain Control Flow, Data Flow, And Side Effects
- Not To Become Assembly Programmers

What Assembly Gives A Reverse Engineer

- Concrete Instruction Sequence
- Register And Memory Updates
- Branch Decisions
- Function Calls And Returns
- Clues About Compiler Decisions

Why Focus On x86-64?

- Very Common In Desktop And Server Software
- Common In Linux Reverse Engineering
- Frequently Seen In CTFs, Malware Analysis, And Native Applications
- Well Supported By Tools Such As Ghidra And `objdump`

Other Architectures Exist

- ARM And AArch64
- MIPS
- RISC-V
- PowerPC

This Module Focuses On One Practical Starting Point: `x86-64`

Basic Structure Of An Assembly Instruction

Many Assembly Instructions Have This Shape:

```
opcode operand1, operand2
```

- Opcode
 - The Operation Such As `mov`, `add`, Or `cmp`
- Operands
 - The Values Or Locations The Instruction Uses

Two Assembly Syntax Styles

- AT&T Syntax
- Intel Syntax

Both Describe The Same Instructions.

The Surface Notation Is Different.

Why We Use Intel Syntax

- Ghidra Uses Intel Syntax For x86 By Default
- More Intuitive

Intel Vs AT&T At A Glance

Intel:

```
mov rax, rbx
```

AT&T:

```
movq %rbx, %rax
```

Both Instructions Copy The Value In `rbx` Into `rax`.

Syntax Difference Summary 1

Intel Syntax:

- Usually Writes `destination, source`
- Uses Plain Register Names Like `rax`
- Often Omits Size Suffixes When The Operand Size Is Clear

AT&T Syntax:

- Usually Writes `source, destination`
- Prefixes Registers With `%`
- Often Uses Instruction Suffixes Such As `b`, `w`, `l`, `q`

Syntax Difference Summary 2

Intel Memory Style:

```
[rax+8]
```

AT&T Memory Style:

```
8(%rax)
```

Assembly From Two Directions

- Compile Source Into Assembly
- Disassemble A Binary Into Assembly

These Are Closely Related But Not Identical Teaching Situations.

For Reverse Engineering, Start From The Binary

- The Binary Is What We Actually Analyze
- Disassembly Is Recovered From Machine Code
- There May Be No Source Code Available
- Tool Output May Reflect Compiler Choices, Not Programmer Style

One Running Demo

This Module Uses One Small C Program With:

- Addition And Comparison
- A Loop
- Array Access
- Character Checking
- Bit Checking
- A Library Call To `puts`

Demo Source: `pattern_demo.c`

```
int sum_and_check(int a, int b) { ... }  
int loop_accumulate(void) { ... }  
int read_array_value(const int *array, int index) { ... }  
int classify_char(const char *s) { ... }  
int main(void) { ... puts("branch hit"); }
```

- Small Enough To Hold In Working Memory
- Rich Enough To Produce Common Reverse-Engineering Patterns

Demo Build And Inspection Commands

```
gcc -O0 -masm=intel -S src/pattern_demo.c -o pattern_demo.s  
gcc -O0 src/pattern_demo.c -o pattern_demo  
objdump -d -M intel pattern_demo
```

- `-masm=intel` Gives Intel Syntax In Generated Assembly
- `objdump -M intel` Makes Disassembly Easier To Read

One Important Caveat

- Generated Assembly And Disassembly Are Related
- But Reverse Engineers Usually Read Disassembly
- Disassembly Comes From Machine Code, Not From Source

So Our Real Target Skill Is Reading What The Binary Preserves.

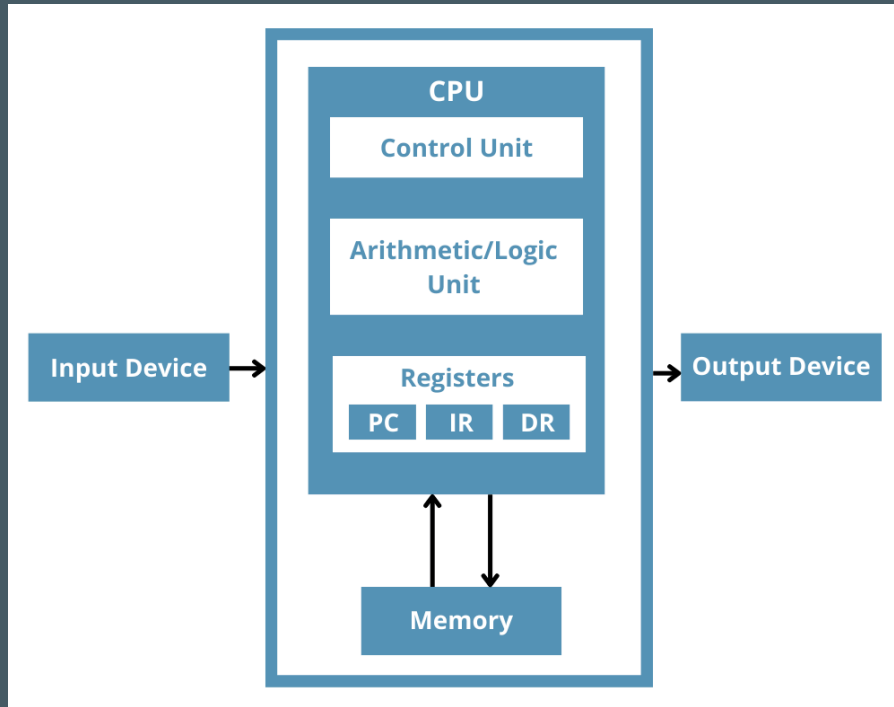
A Simple Computer Model

Think In Terms Of:

- CPU
- Registers
- Memory

This Is Enough For A First Reverse-Engineering Model.

A Simple Computer Model Diagram



Source: [Total Phase, "What is a Register in a CPU and How Does it Work?"](#)

CPU, Registers, And Memory

- CPU Executes Instructions
- Registers Hold Small Fast Working Values
- Memory Holds Larger Program State
- Instructions Move Data Between Registers And Memory
- Instructions Compute Using Values in Registers And Memory

The Stack

- A Special Memory Region Used During Function Execution
- Often Holds Return Addresses
- Often Holds Saved Registers
- Often Holds Local Variables

Important Registers:

- `rsp`
- `rbp`

Common General-Purpose Registers

```
rax, rbx, rcx, rdx  
rsi, rdi  
r8, r9, r10, r11  
r12, r13, r14, r15
```

You Do Not Need To Memorize All Roles On Day One.

Registers Worth Remembering Early

- `rax`
 - Often Used For Return Values
- `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`
 - Often Hold Early Function Arguments
- `rsp`
 - Stack Pointer
- `rbp`
 - Often Used As A Frame Reference

Register Sizes

```
rax    ; 64-bit  
eax    ; lower 32 bits  
ax     ; lower 16 bits  
al     ; lower 8 bits
```

Same Physical Register, Different Width Views.

A Practical Register-Size Fact

```
mov eax, 0
```

- Clears `eax`
- Also Zeroes The Upper 32 Bits Of `rax`

This Appears Constantly In Real Compiler Output.

Very Light Calling Convention

For Many Simple x86-64 Linux Examples:

- First Arguments Often Start In `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`
- Return Value Often Comes Back In `rax`

This Is Enough For This Module.

What `call` Usually Means Practically

When You See A Call:

- Some Arguments Were Prepared Earlier
- Control Transfers To Another Function
- A Return Value May Come Back In `rax`

We Only Need A Light Working Model Today.

Intel Syntax

Intel Syntax Uses:

```
instruction destination, source
```

Example:

```
mov rax, rbx
```

Constants And Immediate Values

Examples:

```
mov eax, 5  
cmp edi, 10  
and eax, 1
```

- 5, 10, And 1 Are Immediate Values

Memory Access Basics

Memory Is Often Referenced With Brackets:

```
[rax]
```

Rough Meaning:

```
Use The Memory At The Address Stored In rax
```


Addressing Form: `[base]`

Example:

```
mov eax, [rax]
```

Common Interpretation:

- Pointer Dereference
- Read A Value From Memory

Addressing Form: `[base + offset]`

Example:

```
mov eax, [rbp-4]  
mov eax, [rax+8]
```

Often Means:

- Local Variable
- Struct Field
- Value At A Fixed Offset

Addressing Form:

$$[\text{base} + \text{index} * \text{scale} + \text{offset}]$$

General Pattern:

$$[\text{base} + \text{index} * \text{scale} + \text{offset}]$$

Examples:

$$\begin{aligned} &[\text{rax} + \text{rbx} * 4] \\ &[\text{rax} + \text{rbx} * 4 + 16] \end{aligned}$$

Often Means Array Or Table Access.

mov

Representative Examples:

```
mov eax, 5  
mov eax, ebx  
mov eax, [rbx]  
mov [rbx], eax
```

- Immediate To Register
- Register To Register
- Memory To Register
- Register To Memory

lea

lea Often Acts Like Arithmetic:

```
lea eax, [rdi+rdi*2]
```

Possible Meaning:

```
eax = rdi * 3;
```

It Does Not Mean "Just Load Address" In Many Compiler Outputs.

add

Example:

```
add eax, ebx
```

Meaning:

```
eax = eax + ebx
```

- Adds Values
- Often Used In Counters, Sums, And Index Updates

sub

Example:

```
sub eax, 4
```

Meaning:

```
eax = eax - 4
```

- Subtracts Values
- Often Used In Differences, Bounds, And Stack Adjustment

imul

Example:

```
imul eax, edi, 3
```

Meaning:

```
eax = edi * 3
```

- Signed Multiplication
- Often Appears In Scaling And Numeric Computation

inc

Example:

```
inc eax
```

Meaning:

```
eax = eax + 1
```

- Increments By One
- Common In Loop Counters

dec

Example:

```
dec eax
```

Meaning:

```
eax = eax - 1
```

- Decrements By One
- Common In Countdown Loops

neg

Example:

```
neg eax
```

Meaning:

```
eax = -eax
```

- Arithmetic Negation
- Flips A Value's Sign

and

Example:

```
and eax, 1
```

Meaning:

```
eax = eax & 1
```

- Masks Bits
- Common For Bit Checks Such As Odd Or Even

or

Example:

```
or eax, 8
```

Meaning:

```
eax = eax | 8
```

- Sets Bits
- Common For Combining Flag Values

xor

Example:

```
xor eax, eax
```

Meaning:

```
eax = 0
```

- Often Used To Clear A Register
- Also Used For General Bitwise Exclusive-Or

not

Example:

```
not eax
```

Meaning:

```
eax = ~eax
```

- Flips Every Bit
- Less Common Than `and`, `or`, Or `xor`, But Still Useful To Recognize

shl

Example:

```
shl eax, 1
```

Meaning:

```
eax = eax << 1
```

- Shifts Bits Left
- Often Similar To Multiplying By Powers Of Two

shr

Example:

```
shr eax, 1
```

Meaning:

```
eax = eax >> 1
```

- Logical Right Shift
- Fills High Bits With Zero

sar

Example:

```
sar eax, 1
```

Meaning:

```
eax = arithmetic_right_shift(eax, 1)
```

- Arithmetic Right Shift
- Preserves The Sign Bit
- Important For Signed Values

Why Side Effects Matter

Some Instructions Matter Because They Change:

- Registers
- Memory
- CPU Flags

Key Questions:

1. What Value Changed?
2. What Flags Changed?
3. What Depends On Those Changes?

CPU Flags We Care About

For This Module, Focus On:

- **ZF** Zero Flag
- **SF** Sign Flag
- **CF** Carry Flag
- **OF** Overflow Flag

Branches Often Depend On These.

cmp

cmp Compares Two Values By Setting Flags.

Example:

```
cmp eax, 10  
je label  
jl other_label
```

cmp Usually Prepares For A Conditional Jump.

test

test Performs A Bitwise and For Flag Setting Only.

Example:

```
test eax, eax  
je    zero_case
```

Very Common Meanings:

- Check For Zero
- Check Whether A Bit Is Set

cmp Vs test

cmp

- Compare Two Potentially Different Values

test

- Check Bit Patterns Or Zero-ness

Both Usually Exist To Set Up A Branch.

Most Instructions Run Sequentially

- Most Instructions Such As `mov`, `add`, `sub`, `and`, `or`, `xor`, And `test` Execute Sequentially: After One Instruction Finishes, The CPU Normally Continues To The Next Instruction
- Some Instructions Change Control Flow Instead:
 - `jmp` And Conditional Jumps Such As `je` Or `jne`
 - `call` And `ret`
 - Software Interrupt Such As `int`

jmp

Example:

```
jmp done
```

Meaning:

Always Jump To done

- Unconditional Jump
- Common For Skipping Code Or Reaching A Shared Exit Path

je

Example:

```
cmp eax, 10  
je  equal_case
```

Meaning:

Jump If Equal

- Usually Depends On **ZF**
- Common After **cmp** Or **test**

jne

Example:

```
cmp eax, 10  
jne not_equal_case
```

Meaning:

Jump If Not Equal

- Usually Means The Compared Values Differ
- Very Common In `if` And Loop Conditions

jg

Example:

```
cmp eax, ebx  
jg  greater_case
```

Meaning:

Jump If Greater

- Signed Comparison
- Use This When The Values Are Interpreted As Signed Integers



Example:

```
cmp eax, ebx  
j1  less_case
```

Meaning:

Jump If Less

- Signed Comparison
- Common In Source Conditions Like `if (a < b)`

jge

Example:

```
cmp eax, ebx  
jge at_least_case
```

Meaning:

Jump If Greater Or Equal

- Signed Comparison
- Common In Bound Checks And Loop Conditions

jle

Example:

```
cmp eax, ebx  
jle at_most_case
```

Meaning:

Jump If Less Or Equal

- Signed Comparison
- Often Appears In Simple **if** Statements



Example:

```
cmp eax, ebx  
ja  above_case
```

Meaning:

Jump If Above

- Unsigned Comparison
- Often Used In Length, Size, Or Index Comparisons



Example:

```
cmp eax, ebx  
jb  below_case
```

Meaning:

Jump If Below

- Unsigned Comparison
- Often Used In Length, Size, Or Index Comparisons

Signed Vs Unsigned Jumps

Signed Examples:

```
j1  
jg  
jle  
jge
```

Unsigned Examples:

```
jb  
ja
```

push

Example:

```
push rax
```

Meaning:

Save rax On The Stack

- Moves `rsp`
- Writes A Value To The Top Of The Stack

pop

Example:

```
pop rax
```

Meaning:

Restore A Value From The Stack Into rax

- Reads From The Top Of The Stack
- Moves `rsp`

`call` And `ret`

```
call puts  
ret
```

- `call` Transfers Control To Another Function And Saves The Return Address
- `ret` Uses That Saved Address To Return Control To The Caller

This Is Enough For A First Reverse-Engineering Pass.

leave

Example:

```
leave  
ret
```

Meaning: Tear Down The Current Stack Frame

- Often Equivalent To `mov rsp, rbp` Followed By `pop rbp`
- Common Near The End Of Unoptimized Functions
- Often Appears Right Before `ret`

Function Prologue And Epilogue

Common Unoptimized Pattern:

```
push rbp  
mov rbp, rsp  
...  
pop rbp  
ret
```

This Often Marks Function Boundaries And Stack-Frame Setup.

Recognizing **if** Patterns

Common Shape:

```
cmp eax, 10  
jle skip  
...  
skip:
```

Often Corresponds To A Source-Level Conditional.

Recognizing Loop Patterns

Look For:

- Initialization
- Comparison
- Back Edge Jump
- Induction Variable Update

These Are Strong Loop Clues In Disassembly.

Recognizing Array And Pointer Patterns

Look For Addressing Like:

```
[rax+rbx*4]
```

Often Means:

- `array[index]`
- Pointer Arithmetic
- Table Lookup

Recognizing Character Checks

Examples Often Involve:

- Loading A Byte
- Comparing Against ASCII Ranges
- Branching Based On The Result

This Is Common In Parsers And Input Validation.

Recognizing Bit Checks

Examples Often Involve:

```
and eax, 1  
test eax, eax
```

Possible Source Meanings:

- Check Odd Or Even
- Check A Flag Bit
- Check A Small Permission Mask

Suggested Mental Workflow

When Reading A Small Disassembly Block:

1. Identify Registers And Operands
2. Identify Memory References
3. Look For Flag-Setting Instructions
4. Look For Branches
5. Look For Calls
6. Reconstruct Likely High-Level Intent

Reverse Engineering Connection

- Assembly Shows What The Binary Actually Does
- Registers, Memory, And Flags Explain Control Decisions
- Common Patterns Reveal `if`, Loops, Arrays, And Checks
- We Read Assembly To Interpret Behavior, Not To Write Programs

Strong Reference

- Intel® 64 And IA-32 Architectures Software Developer's Manual
- Definitive Intel Reference For Instruction Semantics And Architecture Details
- <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

Interactive Analysis Workflow

By Dr. Junjie Zhang

Learning Outcomes

- Apply A Repeatable Ghidra Workflow To Small Binary Challenges
- Rename Functions, Variables, And Types To Improve Understanding
- Distinguish Easy Symbol-Rich Cases From Stripped-Binary Workflows
- Summarize Why A Candidate Input Succeeds Or Fails

Lecture Roadmap

1. Workflow Mindset
2. Repeated Ghidra Moves
3. Crackme Progression

Purpose Of This Module

- Move From Tool Familiarity To Practical Analysis Workflow
- Build Confidence Through A Staged Crackme Sequence
- Practice Recovering Meaning Instead Of Only Reading Syntax

A Simple Analysis Loop

1. Identify Program Entry Clues
2. Rename And Retype To Reduce Ambiguity
3. Test A Hypothesis About Success And Failure
4. Record The Reasoning, Not Only The Answer

Repeated Ghidra Moves

- Look At `main` First When It Is Available
- Rename Functions And Variables Early
- Leverage Imported Functions
- Change Types When Hex-Or Pointer Presentation Hides Meaning
- Use The Decompiler As A Starting Point, Not An Oracle

Where To Get The Executables

- Original Workshop Repository:
 - [Maijin/radare2-workshop-2015](https://github.com/Maijin/radare2-workshop-2015)
- An Alternative Folder:
 - [jzhang369/radare2-workshop-2015/tree/master/IOLI-crackme](https://github.com/jzhang369/radare2-workshop-2015/tree/master/IOLI-crackme)

Structs In Binaries

By Dr. Junjie Zhang

Learning Outcomes

- Explain Why A Struct Is A Composite Data Type
- Explain Why Padding And Alignment Affect Field Offsets
- Recognize Evidence That A Binary Is Using A Struct
- Distinguish Hard Single-Record Cases From Easier Record-Array Cases
- Add New Struct Definitions In Ghidra
- Apply New Struct Definitions To Recovered Data
- Explain Why Library-Defined Structs Often Help Ghidra

Lecture Roadmap

1. Primitive Types To Composite Types
2. Padding And Layout
3. Known Library Structs
4. Source-Defined Structs
5. Hard Vs Easier Recovery Cases

What We Already Know

Earlier Modules Have Already Used Primitive Types Such As:

- `char`
- `short`
- `int`
- Pointers

These Explain Individual Values.

What We Need Next

- Real Programs Group Fields Together
- One Value Is Often Not Enough
- Reverse Engineers Need To Recover Records, Not Only Scalars

So We Now Shift From Primitive Types To Composite Types.

What Is A Composite Data Type?

- A Primitive Type Often Answers: "What Is One Value?"
- A Composite Type Often Answers: "What Fields Belong Together?"
- A Struct Groups Several Fields Into One Record

This Is A Data-Organization Problem As Much As A Syntax Problem.

Structs As Composite Data Types

A Struct May Contain:

- Different Primitive Types
- Different Access Widths
- Different Offsets Inside One Record

From The Binary Side, The Key Clue Is Repeated Access To One Base Address With Stable Offsets.

What Clues Suggest A Struct?

- One Base Object Or Pointer Reused Repeatedly
- Nearby Offsets Accessed Again And Again
- Different Field Sizes At Different Offsets
- The Same Offsets Reappearing Across Multiple Code Paths

Padding Demo First

Use `padding_demo` To Explain:

- What A Struct Layout Is
- Why Field Order Matters
- Why Padding Appears
- Why Total Size Can Surprise You

Running Example: **RecordEntry**

```
typedef struct RecordEntry {  
    char kind;  
    int weight;  
    short delta;  
} RecordEntry;
```

- Small Enough To Inspect Manually
- Rich Enough To Show Padding
- Reused In The Source-Defined Struct Demo

Why Padding Matters

Students Often Expect:

- `kind` Then `weight` Then `delta`
- No Gaps

Compilers Often Insert Padding To Satisfy Alignment Rules.

One Possible Layout

- `kind` At Offset `0`
- `weight` At Offset `4`
- `delta` At Offset `8`
- `sizeof(RecordEntry)` May Be `12`

Source Order Does Not Mean Tightly Packed Layout.

Demo: Showing Padding Directly

In `padding_demo`:

- Print Total Struct Size
- Print Field Sizes
- Print Field Addresses
- Print Field Offsets With `offsetof`
- Compare Multiple Struct Definitions Side By Side

More Than One Layout

The Padding Demo Compares:

- `RecordEntry`
- `CompactRecord`
- `WideHeader`

This Makes It Easier To Show That Similar Fields Can Produce Different Layouts.

Known Header Files Help

Use `libpcap_demo` To Explain Structs From:

- Known Header Files
- Shared Or Dynamically Linked Libraries
- Imported APIs With Existing Type Information

This Is Often The Easier Reverse-Engineering Situation.

Lightweight `libpcap` Story

Keep The Story Simple:

1. Capture Packets From An Interface
2. Decide Whether A Packet Is TCP
3. Parse TCP-Relevant Fields

The Point Is Type Recovery, Not Full Packet Analysis.

Why `libpcap` Is Easier

- The Header Files Already Define Meaningful Layouts
- Imported Calls Provide Semantic Context
- Known Library Types Give Ghidra A Better Starting Point

Source-Defined Structs Are Harder

Why?

- No Helpful Library Signature
- No External Semantic Context
- No Imported Type Metadata
- Less Semantic Context

So The Analyst Must Infer Structure From Access Patterns.

Hard Case: Single Record

If A Built-In Struct Is Used As One Record Only:

- There May Be Only One Base Object
- Only A Few Field Accesses May Appear
- The Evidence May Look Like Unrelated Scalars

This Is Often Extremely Hard.

Source-Defined Demo Functions

- `score_global_single_record`
- `score_stack_single_record`
- `score_heap_single_record`
- `score_global_record_array`
- `score_stack_record_array`
- `score_heap_record_array`

Easier Case: Record Arrays

If The Same Struct Is Enumerated In A Loop:

- Field Offsets Repeat Across Elements
- Record Stride Becomes Visible
- Repetition Confirms The Hypothesis
- Loops Provide Stronger Evidence

Why Record Arrays Are Easier

- Repeated Base-Plus-Offset Access
- Same Fields Used Over And Over
- Enumeration Makes Structure Visible
- Ghidra Retyping Usually Pays Off Faster

This Is Why Repetition Is Such Valuable Evidence.

Compare The Two Worlds

Known Header / Library Case:

- Easier
- Better Type Context
- Better Tool Support

Source-Defined `RecordEntry` Case:

- Harder
- Less Naming Help
- More Inference Required

Ghidra Workflow Emphasis

- Rename Ambiguous Variables
- Create Or Refine Struct Definitions
- Apply Struct Types To Pointers And Globals
- Compare Decompiler Output Before And After Retyping

Key Takeaways

- Structs Are Composite Data Types
- `padding_demo` Explains Real Layout And Padding
- `libpcap_demo` Shows Structs From Known Headers And Libraries
- `struct_demo` Shows Source-Defined Structs
- Single Records Are Harder To Recover Than Enumerated Record Arrays
- Library-Defined Structs Often Give Ghidra A Head Start

Ghidra Scripting: Introduction to Ghidra Programming

By **Dr. Junjie Zhang**

Why Ghidra Scripting Matters?

Ghidra scripting allows reverse engineers to automate common analysis tasks, reduce manual work, and explore deeper program semantics using custom scripts. With scripting, users can create:

- Custom automation pipelines,
- Interactive or headless batch analyses,
- Data extraction and transformation routines,
- Extensions of Ghidra's capabilities.

The scripting environment is built on top of Java and Jython (Python for Java).

Scripting Modes in Ghidra

Ghidra scripts can be written and executed using multiple scripting modes:

Interactive (GUI-Based)

This is the default scripting mode within the Ghidra GUI. You can create, edit, and run scripts via the **Script Manager**. The context includes the currently open program and address of the cursor.

Pros:

- Immediate feedback and testing,
- Direct GUI integration (e.g., `currentAddress`, highlights).

Headless Mode (Command Line)

Ghidra provides **`analyzeHeadless`**, a command-line interface to run scripts on one or more binaries outside of the GUI.

```
analyzeHeadless <ProjectDir> <ProjectName> -import <Binary> -scriptPath  
<PathToScripts> -postScript <YourScript> [args...]
```

Pros:

- Batch processing,
- Integration with automation pipelines,
- No GUI overhead.

Third-Party Interfaces (e.g., PyGhidra)

Pyhidra is a Python library that provides direct access to the Ghidra API.

This mode is useful when integrating Ghidra with Python-based tooling such as Jupyter notebooks or static analysis pipelines.

How Scripting Works in Ghidra

Every script you write in Ghidra extends the **GhidraScript** class. The **GhidraScript** class extends the **FlatProgramAPI** class. Their relationship is characterized as follows.

```
Your Script
└ inherits → GhidraScript
               └ extends → FlatProgramAPI
```

- **FlatProgramAPI** internally holds a reference to the current ProgramDB object and offers numerous methods for interacting with binary constructs such as memory, functions, instructions, addresses, and symbols.
- **GhidraScript** is the abstract base class for scripting in Ghidra. It extends FlatProgramAPI, inheriting its analysis methods. It additionally provides interactive utilities such as askString, askAddress, and access to monitor for progress reporting and cancellation.

Example Script Skeleton

```
# Ghidra Scripting Description: Our First Script
# @category: GhidraScripting
# @author: Junjie Zhang

print("Hello World!")
```

Load this folder into Ghidra.

Summary

- Ghidra scripting provides flexible, programmable analysis of binaries.
- You can write and run scripts in GUI or headless mode.
- Ghidra also supports third-party integration with libraries like **Pyghidra**.

Ghidra Scripting: Working with Functions

By **Dr. Junjie Zhang**

Why Functions Matter?

Functions are fundamental units of modularity and abstraction in software development. They enable developers to decompose complex systems into smaller, manageable components, facilitating

- Code reuse,
- Readability,
- Maintainability, and
- Testing.

In the context of software reverse engineering, functions serve as critical analysis targets. Typical function-oriented reverse engineering tasks include, but are not limited to,

- Recover function boundaries,
- Infer meaningful names,
- Reconstruct signatures (including return types and parameters), and
- Understand the logic implemented within each function.

The indepth understanding of functions is essential for tasks such as vulnerability discovery, malware analysis, and binary lifting.

Function analysis can generally be categorized into two strategies:

- **Intra-Procedural Analysis:** This approach focuses on analyzing a single function in isolation, without considering its interactions with other functions. Common tasks include:
 - Inferring function names
 - Generating control flow graphs (CFGs)
 - Performing similarity analysis
 - Conducting taint analysis
 - Examining data flow within the function
 - And more
- **Inter-Procedural Analysis:** This strategy involves analyzing one or more functions while taking into account their calling contexts and interactions with other functions. Typical applications include:
 - Building call graphs
 - Performing whole-program taint analysis
 - Executing whole-program data flow analysis
 - Applying symbolic execution across function boundaries
 - And more

In this lecture, we will focus on foundational scripting skills in Ghidra to support intra-procedural analysis. Specifically, we will work on syntactic analysis techniques rather than deeper semantic analyses such as taint tracking or data flow analysis.

Where to Find More Information?

Most relevant documentation for working with functions in Ghidra can be found in the official Javadoc at:

/docs/GhidraAPI_javadoc/api/ghidra/program/model/listing/Function.html

This resource provides detailed descriptions of the Function class and its methods, which are essential for scripting and analyzing functions programmatically.

Working with Functions

Function Layout in a Binary

In a typical binary executable, the `.text` section contains the machine code for all defined (non-inline) functions. These functions are generally laid out in a **sequential (or consecutive)** manner in memory. That is, one function usually begins immediately after the previous one ends, though this layout may include padding to satisfy alignment constraints. For example, if one function ends at address 0x1000, the next might begin at 0x1010 depending on the required alignment.

```
block-beta
  block
    columns 1
    a["Function 1"]
    b["Function 2"]
    c["....."]
    d["Function N"]
  end
```

Retrieving One Function Object in Ghidra

Ghidra's `FlatProgramAPI` provides several convenient methods to access Function objects within a binary. These methods allow you to retrieve specific functions based on their name, address, or relative position:

- `getFirstFunction()` / `getLastFunction()` : Retrieves the first or last function in the program.
- `getFunction(String name)` : Returns a function by its name, if it exists.
- `getFunctionAt(Address entryPoint)` : Retrieves a function located exactly at the given entry point address.
- `getFunctionContaining(Address address)` : Returns the function that contains the specified address (e.g., the address of an instruction or variable).
- `getFunctionAfter(Address address)` : Gets the next function after the specified address.
- `getFunctionBefore(Address address)` : Gets the previous function before the specified address.

These methods provide the foundation for navigating and analyzing functions through scripting in Ghidra.

```
# Get a function using getFirstFunction()
func = getFirstFunction()
print(func)
```

```
# Get a function using getFunction(String name)
name = "xyz"
func = getFunction(name)
if func is not None:
    print(func)
else:
    print("Function with {} does not exist.".format(name))
```

```
# Get a function using getFunctionAt(Address address)
addr = currentAddress # currentAddress is the address indicated by your cursor.
func = getFunctionAt(addr)
if func is not None:
    print(func)
else:
    print("Function with {} as the entry point does not exist.".format(addr))
```

```
# Get a function using getFunctionContaining(Address address)
addr = currentAddress # currentAddress is the address indicated by your cursor.
func = getFunctionContaining(addr)
if func is not None:
    print(func)
else:
    print("Function containing {} does not exist.".format(addr))
```

Enumerating all Functions in a Binary

The methods in `FlatProgramAPI` can be used to enumerate all functions in a binary.

- `getFirstFunction()` / `getLastFunction()` : Retrieves the first or last function in the program.
- `getFunctionAfter(Address address)` : Returns the next function in memory after the specified address. Even if the address lies within a function, this method returns the one that starts *after* it.
- `getFunctionAfter(Function function)` : Returns the next function in memory after the entry point of the given function. This is helpful for forward traversal of functions based on layout in memory.
- `getFunctionBefore(Address address)` : Returns the last function that starts before the given address. If the address lies inside a function, the method still returns the one that starts before it.
- `getFunctionBefore(Function function)` : Returns the function whose entry point is located just before the entry point of the provided function. This is useful for backward traversal of functions.

```
# Enumerate all functions using getFunctionAfter(Function function)
func = getFirstFunction()
while func:
    print(func)
    func = getFunctionAfter(func)
```

```
# Enumerate all functions using getFunctionBefore(Function function)
func = getLastFunction()
while func:
    print(func)
    func = getFunctionBefore(func)
```

In addition to use these methods in `FlatProgramAPI`, you can also use `FunctionManagerDB` class. A `FunctionManagerDB` object can be returned by the `getFunctionManager()` method in defined the `ProgramDB` class. Again, `currentProgram` is an object of the `ProgramDB` class.

```
# Enumerate all functions using FunctionManagerDB
# Ghidra Scripting: Functions
# @category: GhidraScripting
# @author: Junjie Zhang

myFuncManager = currentProgram.getFunctionManager()
allFuncs = myFuncManager.getFunctions(True)
for i in allFuncs:
    print(i)
```

Retrieving Useful Information from a Function Object

- **Function Name**
- **Calling Convention**
 - `__cdecl`: Caller cleans the stack; Arguments passed right-to-left
 - `__stdcall`: Callee cleans the stack; Arguments passed right-to-left
 - `__fastcall`: Callee cleans the stack; First 1–2 arguments passed in registers (e.g., ECX, EDX); Remaining arguments on stack (right-to-left)
 -
- **Function Signature**
 - Return type
 - Parameters and their corresponding types
- **Entry Point**: The address of the first instruction in the function.
- **Exit Point**: The address of the last instruction in the function.

- **Function Size:** The number of bytes that comprise the function body.
- **Function Type**
 - *Internal function:* a function whose implementation resides within the binary being analyzed.
 - *External function:* a function that is declared but not implemented in the current binary; it is typically implemented in another library
 - *Inline function:* a function whose code has been copied directly into the caller during compilation, so it does not appear as a distinct function in the binary.
 - *Thunk function:* a function corresponds to a fragment of code which simply passes control to a destination function (e.g., for DLL import resolution).
- **Parameters**
 - Retrieve the name and type of each parameter
 - Modify the name and type of each parameter
- **Local Variables**
 - Retrieve the name and type of each local variable
 - Modify the name and type of each local variable
- **Return Variable**
 - Retrieve the name and type of the return variable

In addition to retrieving the information, you can also set certain information such as

- The variable name and the type,
- The calling convention, and
- The comment.

```
# Ghidra Scripting: Functions
# @category: GhidraScripting
# @author: Junjie Zhang

myFuncManager = currentProgram.getFunctionManager()
allFuncs = myFuncManager.getFunctions(True)
for f in allFuncs:

    f_body = f.getBody()

    print("-"*10)
    print("name:\t\t{}".format(f.getName()))
    print("calling convention:\t\t{}".format(f.getCallingConventionName()))
    print("signature:\t\t{}".format(f.getSignature()))
    print("entry address:\t\t{}".format(f.getEntryPoint()))
    print("exit address:\t\t{}".format(f_body.getMaxAddress()))
    print("size of function body:\t\t{}".format(f_body.getNumAddresses()))
    print("internal function:\t\t{}".format(not f.isExternal()))
    print("external function:\t\t{}".format(f.isExternal()))
```

```
print("inline function:\t\t{}".format(f.isInline()))  
print("thunk function:\t\t{}".format(f.isThunk()))
```

Ghidra Scripting: Working with Instructions

By **Dr. Junjie Zhang**

Why Instructions Matter?

In Ghidra, **instructions** represent the low-level operations executed by a processor and are fundamental to binary analysis. While binaries contain raw machine code, Ghidra disassembles this code into *human-readable assembly instructions*, enabling analysts to understand program behavior at the instruction level.

In this section, the term *instructions* specifically refers to assembly instructions. Later, we will introduce intermediate representations, such as **Pcode**, which can also be considered a form of instruction but at a higher abstraction level.

Ghidra provides APIs for interacting with instructions programmatically, supporting a variety of tasks such as:

- Iterating over instructions
- Extracting mnemonics and operands
- Identifying relationships with other objects (e.g., through **reference**).

Where to Find More Information?

For more detailed information on programming with instructions in Ghidra, refer to the official Ghidra API documentation. The relevant information can be found in your Ghidra installation at:

`docs/GhidraAPI_javadoc/api/ghidra/program/model/listing/Instruction.html`

Working With Instructions

Retrieving One Instruction Object in Ghidra

Let's begin with the **FlatProgramAPI**, which provides several convenient methods to retrieve **Instruction** objects:

- `getFirstInstruction()` / `getLastInstruction()`
Retrieve the first or last instruction in the entire program.
- `getFirstInstruction(Function function)`
Returns the first instruction within a specified function.
- `getInstructionAt(Address address)`
Retrieves the instruction located exactly at the given address, if one exists.
- `getInstructionContaining(Address address)`
Returns the instruction that contains the specified address (useful for instructions spanning multiple bytes).
- `getInstructionAfter(Address address)` / `getInstructionAfter/Instruction instruction)`
Returns the instruction that follows the given address or instruction.

- `getInstructionBefore(Address address) / getInstructionBefore(Instruction instruction)`

Returns the instruction that precedes the given address or instruction.

```
# get the first instruction and the last instruction in this binary.
firstInstruction = getFirstInstruction()
print(firstInstruction)

lastInstruction = getLastInstruction()
print(lastInstruction)
```

```
# enumerate all functions in this binary, and for each function print out its
first instruction.
fm = currentProgram.getFunctionManager()
allFuncs = fm.getFunctions(True)
for f in allFuncs:
    firstInstruction = getFirstInstruction(f)
    print("The first instruction of {} function is {}".format(f,
firstInstruction))
```

```
# ask the user to offer an address and then attempt to get the instruction at that
address. This address is for the address of the very first byte of the
instruction.
addr = askAddress("Ghidra Scripting - Instructions", "Please give me an address!")
myInstruction = getInstructionAt(addr)
if myInstruction:
    print(myInstruction)
else:
    print("no instruction found at the address of {}".format(addr))
```

```
# ask the user to offer an address and then attempt to get the instruction that
contains address. Note: an instruction may have multiple bytes.
addr = askAddress("Ghidra Scripting - Instructions", "Please give me an address!")
myInstruction = getInstructionContaining(addr)
if myInstruction:
    print(myInstruction)
else:
    print("no instruction that contains the address of {}".format(addr))
```

Enumerating all Instructions in the Binary

Option 1:

You can use `getInstructionAfter()` and `getInstructionBefore()` to enumerate all instructions inside a binary.

```
# Get the first instruction and then enumerate the one after this, and keep doing that.
myInstruction = getFirstInstruction()
while myInstruction:
    print(myInstruction)
    myInstruction = getInstructionAfter(myInstruction)
```

Option 2:

Get the `listing` object, and use the `listing`'s methods.

```
# getInstructions returns an iterator of instructions inside this binary
myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(True)
for inst in instructionIterator:
    print(inst)
```

Enumerating all Instructions in a Function

Option 1:

Enumerate all instructions starting from the first instruction of the function and verify whether each instruction's address is within the function's body.

```
# Ghidra Scripting: Instructions
# @category: GhidraScripting
# @author: Junjie Zhang

# display all instructions inside the function indicated by your cursor.
myFunc = getFunctionContaining(currentAddress)
if myFunc:
    fbody = myFunc.getBody() # this is the body of the function, it is an
    AddressSetView, a set of addresses.
    print(myFunc)
    myInstruction = getFirstInstruction(myFunc)
    while myInstruction and fbody.contains(myInstruction.getAddress()): # verify
        whether the address of this instruction is inside the set of addresses for this
        function.
        print("address: {}, instruction: {}".format(myInstruction.getAddress(),
        myInstruction))
        myInstruction = getInstructionAfter(myInstruction)
else:
    print("No function found at {}".format(currentAddress))
```

Option 2:

Using `getInstructions(AddressSetView addrSet, boolean forward)` of the `listing` object.

```
# Ghidra Scripting: Instructions
# @category: GhidraScripting
# @author: Junjie Zhang

myFunc = getFunctionContaining(currentAddress)

if myFunc:
    print(myFunc)
    fbody = myFunc.getBody() #fbody is an object of AddressSetView
    myListing = currentProgram.getListing()
    instructionIterator = myListing.getInstructions(fbody, True) #
myListing.getInstructions(AddressSetView, Boolean) will return instructions that
are inside this set of addresses
    for inst in instructionIterator:
        print("address: {}, instruction: {}".format(inst.getAddress(), inst))
```

Retrieving Useful Information from an Instruction Object

- mnemonic: `getMnemonicString()`
 - type
- operands: `getInputObjects()`
 - value
 - type
- address: `getAddress()`, `getMinAddress()`, and `getMaxAddress()`
- size: `getLength()`
- Fall-Through: `getFallThrough()` - the fall-through of an instruction refers to the next sequential instruction in memory that would be executed if there is no explicit change in control flow.
 - For most linear instructions (`mov`, `add`, and etc.), execution naturally proceeds to the next instruction.
 - For branching instructions, such as `JMP`, `CALL`, and `RET`, execution may deviate from the sequential flows.
- Flows: `getFlows()` - an array of addresses to be executed other than a fall-through.

```
# Ghidra Scripting: Instructions
# @category: GhidraScripting
# @author: Junjie Zhang

# This example is to count the occurrence for each mnemonic string showing up in
this body of the current function.
myDict = {}

myFunc = getFunctionContaining(currentAddress)
if myFunc:
    fbody = myFunc.getBody() #fbody is an object of AddressSetView
```

```

print(myFunc)
myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(fbody, True)
for inst in instructionIterator:
    mnemonic = inst.getMnemonicString()
    if mnemonic in myDict:
        myDict[mnemonic] = myDict[mnemonic] + 1
    else:
        myDict[mnemonic] = 1

print(myDict)

```

```

# Ghidra Scripting: Instructions
# @category: GhidraScripting
# @author: Junjie Zhang

# get address, min address, max address, and the length of an instruction
# the address is the same to the min address
# the length of an instruction is the number of bytes for this instruction
myFunc = getFunctionContaining(currentAddress)
if myFunc:
    fbody = myFunc.getBody() #fbody is an object of AddressSetView
    print(myFunc)
    myListing = currentProgram.getListing()
    instructionIterator = myListing.getInstructions(fbody, True)
    for inst in instructionIterator:
        operands = inst.getInputObjects()
        print(inst)
        print("address: {}, min_address: {}, max_address: {}, size:
{}".format(inst.getAddress(), inst.getMinAddress(), inst.getMaxAddress(),
inst.getLength()))

```

```

# Ghidra Scripting: Instructions
# @category: GhidraScripting
# @author: Junjie Zhang

# get the fallthrough instructions and non-fallthrough/target instructions for J,
CALL, and RET instructions.

myFunc = getFunctionContaining(currentAddress)
if myFunc:
    print(myFunc)
    fbody = myFunc.getBody() #fbody is an object of AddressSetView
    myListing = currentProgram.getListing()
    instructionIterator = myListing.getInstructions(fbody, True)
    for inst in instructionIterator:

        if inst.getMnemonicString().startswith("J") or
inst.getMnemonicString().startswith("CALL") or

```

```

inst.getMnemonicString().startswith("RET"):

    print("-"*10)

    print("{}\t{}".format(inst.getAddress(), inst))

    addrFallThrough = inst.getFallThrough()
    if addrFallThrough:
        fallThroughInst = getInstructionAt(addrFallThrough)
        print("    fallthrough to: {}\t{}".format(addrFallThrough,
fallThroughInst))

    addrOtherThanFallThrough = inst.getFlows()
    for one in addrOtherThanFallThrough:
        notFallThroughInst = getInstructionAt(one)
        print("    notfallthrough/target/flow to: {}\t{}".format(one,
notFallThroughInst))

```

Applications

Application 1: Enumerate All Callees of the Current Function

Option 1:

1. **Enumerate** all instructions within the function.
2. **Identify** **CALL** instructions.
3. **Parse** each **CALL** instruction to extract the operand, representing the callee's address.
4. **Determine** the function whose entry point matches the extracted address.

```

# Ghidra Scripting: Instructions
# @category: GhidraScripting
# @author: Junjie Zhang

# Enumerate all callees of the current function.

calleeSet = set()

myFunc = getFunctionContaining(currentAddress)
if myFunc:
    print(myFunc)
    fbody = myFunc.getBody() #fbody is an object of AddressSetView
    myListing = currentProgram.getListing()
    instructionIterator = myListing.getInstructions(fbody, True)
    for inst in instructionIterator:

        if inst.getMnemonicString().startswith("CALL"):

            calleeAddress = inst.getOpObjects(0)[0]
            callee = getFunctionAt(calleeAddress)
            calleeSet.add(callee)

```

```
print("{} has following callees:".format(myFunc))

for i in calleeSet:
    print("\t{}".format(i))
```

Option 2:

Using flows via `getFlows()` to obtain the address of the callee eliminates the need to parse instructions for callee addresses.

```
# Ghidra Scripting: Instructions
# @category: GhidraScripting
# @author: Junjie Zhang

# Enumerate all callees of the current function.

calleeSet = set()

myFunc = getFunctionContaining(currentAddress)
if myFunc:
    print(myFunc)
    fbody = myFunc.getBody() #fbody is an object of AddressSetView
    myListing = currentProgram.getListing()
    instructionIterator = myListing.getInstructions(fbody, True)
    for inst in instructionIterator:

        if inst.getMnemonicString().startswith("CALL"):

            addrFlowTo = inst.getFlows() # addrFlowTo is the callee's entry point.
            for addr in addrFlowTo:
                callee = getFunctionAt(addr)
                calleeSet.add(callee)

print("{} has following callees:".format(myFunc))

for i in calleeSet:
    print("\t{}".format(i))
```

Application 2: Enumerate All Callers of the Current Function

Option 1:

- **Enumerate** all instructions in the analyzed binary and identify `CALL` instructions.
- For each `CALL` instruction, **check** whether its operand matches the address of the current function:
 - If it matches, the `CALL` instruction targets the current function, and the function containing this `CALL` instruction is identified as a caller.
 - **Locate** the function that contains the address of the `CALL` instruction.

```

# Ghidra Scripting: Instructions
# @category: GhidraScripting
# @author: Junjie Zhang

# Enumerate all callers of the current function.

callers = set()

myFunc = getFunctionContaining(currentAddress)

if myFunc:
    # getInstructions returns an iterator of instructions inside this binary
    myListing = currentProgram.getListing()
    instructionIterator = myListing.getInstructions(True)
    for inst in instructionIterator:
        if inst.getMnemonicString().startswith("CALL"):
            calleeAddr = inst.getOpObjects(0)[0]
            if myFunc.getEntryPoint() == calleeAddr:
                callerFunc = getFunctionContaining(inst.getAddress())
                if callerFunc:
                    print("Caller: {} at {} calls {}".format(callerFunc,
inst.getAddress(), myFunc))
                    callers.add(callerFunc)
    print("callers of {}".format(myFunc))
    for i in callers:
        print(i)

```

Option 2:

Using `getFlows()` instead of `getOpObjects(0)[0]` enhances code readability by removing the need to manually parse instructions for callee addresses.

```

# Ghidra Scripting: Instructions
# @category: GhidraScripting
# @author: Junjie Zhang

# Enumerate all callers of the current function.

callers = set()

myFunc = getFunctionContaining(currentAddress)

if myFunc:
    # getInstructions returns an iterator of instructions inside this binary
    myListing = currentProgram.getListing()
    instructionIterator = myListing.getInstructions(True)
    for inst in instructionIterator:
        if inst.getMnemonicString().startswith("CALL"):
            addrFlowTo = inst.getFlows()
            for calleeAddr in addrFlowTo:
                if myFunc.getEntryPoint() == calleeAddr:

```

```
        callerFunc = getFunctionContaining(inst.getAddress())
        if callerFunc:
            print("Caller: {} at {} calls {}".format(callerFunc,
inst.getAddress(), myFunc))
            callers.add(callerFunc)

print("callers of {}".format(myFunc))
for i in callers:
    print(i)
```


Ghidra Scripting: Working with References

By **Dr. Junjie Zhang**

Why References Matter?

In Ghidra, a reference consists of two addresses:

- A **FROM** address
- A **TO** address

A **reference** represents a relationship between two addresses and can indicate various interactions, such as:

- Control flow changes
- Function calls
- Data access

References are very useful for automating binary analysis and enhancing reverse engineering efficiency. They provide several key benefits:

- They eliminate the need to parse instructions and interpret the semantic meaning of diverse mnemonics.
- They assist analysts in understanding how instructions and data interact, aiding in the reconstruction of program logic.

An example: <FROM: 00102788, To: 00105b60, Type: Unconditional_Call> because at 0x00102788, there is an instruction **CALL 0x00105b60**

Where to Find More Information?

For more detailed information on programming with references in Ghidra, refer to the official Ghidra API documentation. The relevant information can be found in your Ghidra installation at:

docs/GhidraAPI_javadoc/api/ghidra/program/model/symbol/Reference.html

Working with References

Retrieving Reference Objects

Given an address, such as **addr**, you can identify:

- References where the **FROM** address is **addr**.
- References where the **TO** address is **addr**.

This address can be any location within the binary, such as the address of an instruction, the entry point of a function, or the address of a byte.

Both the **FlatProgramAPI** and **ReferenceManager** classes provide methods to retrieve references associated with a specific address. It is important to note that a single address may be associated with multiple references. A **ReferenceManager** object can be obtained using **currentProgram.getReferenceManager()**.

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang
# enumerate all references from and to an address
addr = askAddress("Ghidra Scripting - References", "Please input an address:")
for i in getReferencesFrom(addr):
    print("a ref from this address: {}".format(i))
for i in getReferencesTo(addr):
    print("a ref to this address: {}".format(i))
```

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang

addr = askAddress("Ghidra Scripting - References", "Please input an address:")

refManager = currentProgram.getReferenceManager()

for i in refManager.getReferencesFrom(addr):
    print("a ref from this address: {}".format(i))
for i in refManager.getReferencesTo(currentAddress):
    print("a ref to this address: {}".format(addr))
```

Enumerating Reference Objects Relevant to a Function

Since a reference is always associated with an address, you can enumerate references relevant to a function by:

1. **Enumerating all addresses** within the function.
2. **Retrieving references** for each address.

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang
myFunc = getFunctionContaining(currentAddress)
if myFunc:
    print(myFunc.getName())
    fbody = myFunc.getBody()
    for addr in fbody.getAddresses(True):
        for i in getReferencesFrom(addr):
            print("a ref from this address {}: {}".format(addr, i))
        for i in getReferencesTo(addr):
            print("a ref to this address {}: {}".format(addr, i))
```

Enumerating Reference Objects Relevant to an Instruction

Again, a reference is always associated with an address. You can obtain the address of an instruction and then retrieve all references from or to that address.

For example, let's get the first instruction of a function and identify references associated with this instruction.

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang
myFunc = getFunctionContaining(currentAddress)
if myFunc:
    inst = getFirstInstruction(myFunc)
    if inst:
        addr = inst.getAddress()
        for i in getReferencesFrom(addr):
            print("a ref from this address {}: {}".format(addr, i))
        for i in getReferencesTo(addr):
            print("a ref to this address {}: {}".format(addr, i))
```

Understanding Reference Types

A reference characterizes the relationship between the **FROM** address and the **TO** address. This relationship can represent various interactions, such as:

- **Function Call:** The instruction at the **FROM** address calls a function whose entry point is the **TO** address.
- **Unconditional Jump:** The instruction at the **FROM** address unconditionally jumps to the instruction at the **TO** address.
- **Conditional Jump:** The instruction at the **FROM** address conditionally jumps to the instruction at the **TO** address.
- **Data Access:** The instruction at the **FROM** address reads/writes data at the **TO** address.
- ... (Additional reference types)

For a reference, you can use the `getReferenceType()` method to obtain a `RefType` object, which indicates the type of the reference. Examples of reference types include, but are not limited to:

- `CONDITIONAL_JUMP`
- `UNCONDITIONAL_JUMP`
- `CONDITIONAL_CALL`
- `UNCONDITIONAL_CALL`
- `COMPUTED_CALL`

The `RefType` class provides several methods to evaluate the type of a reference. For instance:

- Use `isCall()` to determine if the reference is a `CALL` type, regardless of whether it is `CONDITIONAL_CALL`, `UNCONDITIONAL_CALL`, `COMPUTED_CALL`, etc. Similarly, you can use analogous methods to identify `JMP` instructions.
- But if you can always check the specific type of a reference such as `UNCONDITIONAL_JUMP`. You can use `ref.getReferenceType() == RefType.UNCONDITIONAL_JUMP`.

Here's an example: Enumerate all **JMP** references **from** each instruction in a binary, regardless of their specific types, such as **CONDITIONAL_JUMP** or **UNCONDITIONAL_JUMP**.

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang

myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(True)
for inst in instructionIterator:
    addr = inst.getAddress()
    allRefsFromAddr = getReferencesFrom(addr)
    for ref in allRefsFromAddr:
        if ref.getReferenceType().isJump():
            print("{} with the specific type of {}".format(ref,
ref.getReferenceType().getName()))
```

Here's another example: Enumerate all **UNCONDITIONAL_JUMP** references **from** each instruction in a binary.

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang

from ghidra.program.model.symbol import RefType

myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(True)
for inst in instructionIterator:
    addr = inst.getAddress()
    allRefsFromAddr = getReferencesFrom(addr)
    for ref in allRefsFromAddr:
        if ref.getReferenceType() == RefType.UNCONDITIONAL_JUMP:
            print("{} with the specific type of {}".format(ref,
ref.getReferenceType().getName()))
```

Application 1: Enumerate All Callees of the Current Function

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang

#find all callees of the current function.

myFunc = getFunctionContaining(currentAddress)
if myFunc:
    print(myFunc)

    fbody = myFunc.getBody() #fbody is an object of AddressSetView
```

```

myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(fbody, True)

for inst in instructionIterator:
    addr = inst.getAddress()
    for ref in getReferencesFrom(addr):
        if ref.getReferenceType().isCall():
            calleeAddr = ref.getToAddress()
            calleeFunc = getFunctionAt(calleeAddr)
            print("{} at {} calls {}".format(myFunc, addr, calleeFunc))

```

Application 2: Enumerate All Callers of the Current Function

We already implemented two solutions by:

1. Parsing instruction operands, and
2. Using the `getFlow()` method of the `Instruction` class.

For either implementation, we need to enumerate all instructions within the binary. The implementation using `getFlow()` is shown below.

```

# old implementation
callers = set()

myFunc = getFunctionContaining(currentAddress)
if myFunc:
    # getInstructions returns an iterator of instructions inside this binary
    myListing = currentProgram.getListing()
    instructionIterator = myListing.getInstructions(True)
    for inst in instructionIterator:
        if inst.getMnemonicString().startswith("CALL"):
            for calleeAddr in inst.getFlows():
                if myFunc.getEntryPoint() == calleeAddr:
                    callerFunc = getFunctionContaining(inst.getAddress())
                    print("Caller: {} at {} calls {}".format(callerFunc,
inst.getAddress(), myFunc))
                    callers.add(callerFunc)

print(callers)

```

Let's see how `reference` can assist in simplifying the implementation:

- Get the entrypoint of the current function.
- Retrieve references **to** this entrypoint and filter for `CALL` references only.
- Extract the `FROM` addresses from these references.
- For each of these addresses, identify the function that contains the address — this function is a caller.

No need to enumerate all instructions of this binary!

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang

#find all callers of the current function.

myFunc = getFunctionContaining(currentAddress)
if myFunc:
    entrypoint = myFunc.getEntryPoint()
    for ref in getReferencesTo(entrypoint):
        if ref.getReferenceType().isCall():
            callerInstAddr = ref.getFromAddress()
            callerFunc = getFunctionContaining(callerInstAddr)
            print("{} is called by {} at {}".format(myFunc, callerFunc,
            callerInstAddr))
```

Application 3: Identify All Functions with Loop(s)

All you need to do is:

1. Enumerate all instructions within a function.
2. Identify the **CALL** reference **from** each instruction.
3. Check whether the **TO** address:
 - Is within the body of the current function (this can be ignored as it is always the case).
 - Is smaller than the address of the current instruction (indicating a backward jump).

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang

#find all callers of the current function.

funcsWithLoop = set()

myListing = currentProgram.getListing()
fm = currentProgram.getFunctionManager()
allFuncs = fm.getFunctions(True)

for f in allFuncs:
    f_body = f.getBody()

    instructionIterator = myListing.getInstructions(f_body, True)

    for inst in instructionIterator:
        allRefsFromInst = getReferencesFrom(inst.getAddress())
        allJumpRefsFromInst = filter(lambda x: x.getReferenceType().isJump(),
allRefsFromInst)
        allBackwardJumpRefsFromInst = filter(lambda x: x.getFromAddress() > x
.getToAddress(), allJumpRefsFromInst)
        if len(allBackwardJumpRefsFromInst) > 0:
```

```
        funcsWithLoop.add(f)

print("Functions with loop:")
for f in funcsWithLoop:
    print(f)
```

Application 4: Identify All Recursion Functions

To identify recursive functions, check whether any instruction within a function makes a call to the function itself.

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang

#find all recursion functions.

funcsWithRecursion = set()

myListing = currentProgram.getListing()
fm = currentProgram.getFunctionManager()
allFuncs = fm.getFunctions(True)

for f in allFuncs:
    f_body = f.getBody()

    instructionIterator = myListing.getInstructions(f_body, True)

    for inst in instructionIterator:
        allRefsFromInst = getReferencesFrom(inst.getAddress())
        allCallRefsFromInst = filter(lambda x: x.getReferenceType().isCall(),
allRefsFromInst)
        allSelfCallRefsFromInst = filter(lambda x: x.getToAddress() ==
f.getEntryPoint(), allCallRefsFromInst)
        if len(allSelfCallRefsFromInst) > 0:
            funcsWithRecursion.add(f)

print("Functions with recursion:")
for f in funcsWithRecursion:
    print(f)
```

Can you figure out a simpler solution using references?

```
# Ghidra Scripting: References
# @category: GhidraScripting
# @author: Junjie Zhang

#find all recursion functions.
```

```
funcsWithRecursion = set()

myListing = currentProgram.getListing()
fm = currentProgram.getFunctionManager()
allFuncs = fm.getFunctions(True)

for f in allFuncs:

    entryPoint = f.getEntryPoint()
    f_body = f.getBody()

    allRefsToEntryPoint = getReferencesTo(entryPoint)
    allCallRefsToEntryPoint = filter(lambda x: x.getReferenceType().isCall(),
allRefsToEntryPoint)
    allSelfCallRefsToEntryPoint = filter(lambda x:
f_body.contains(x.getFromAddress()), allCallRefsToEntryPoint)

    if len(allSelfCallRefsToEntryPoint) > 0:
        funcsWithRecursion.add(f)

print("Functions with recursion:")
for f in funcsWithRecursion:
    print(f)
```


Ghidra Scripting: Working with Basic Blocks

By **Dr. Junjie Zhang**

Why Basic Blocks Matter?

A **basic block** is a sequence of consecutive instructions in a program with the following characteristics:

- **Single Entry Point:** Execution always starts at the first instruction of the block and cannot be entered at any other point within the block.
- **Single Exit Point:** Execution always exits from the last instruction of the block, either through a jump, or branch.
- **Linear Flow:** There are no jumps or branches within the block itself, except at the end.

It is worth noting that when constructing basic blocks, function calls are generally treated as regular instructions rather than as jumps or branches.

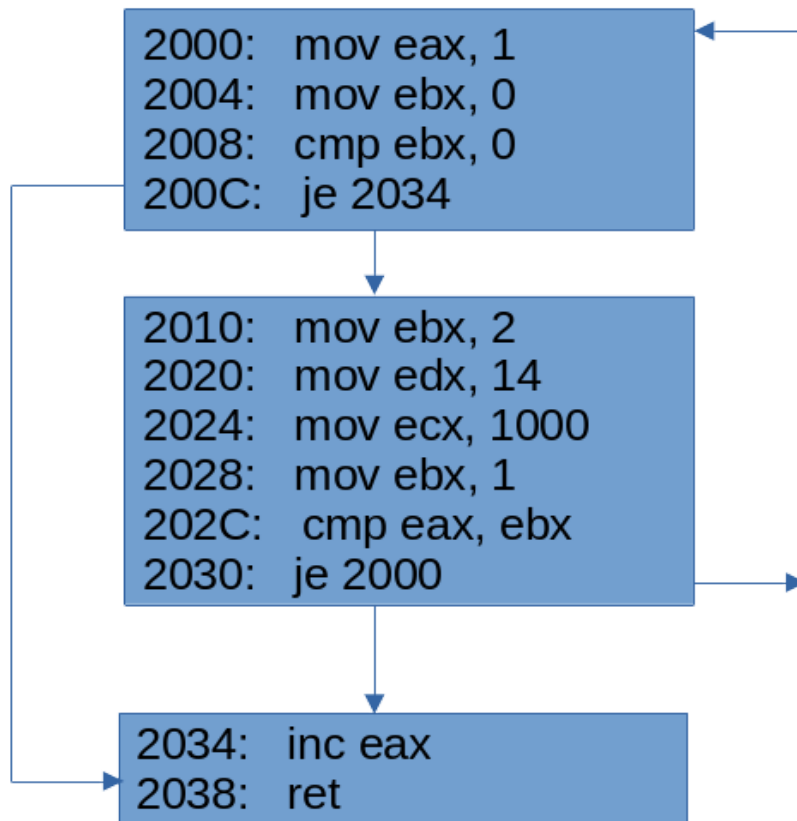
Example:

```
2000:  mov eax, 1
2004:  mov ebx, 0
2008:  cmp ebx, 0
200C:  je 2034
2010:  mov ebx, 2
2020:  mov edx, 14
2024:  mov ecx, 1000
2028:  mov ebx, 1
202C:  cmp eax, ebx
2030:  je 2000
2034:  inc eax
2038:  ret
```

For this example, we have three basic blocks.

```
;Basic Block 1
2000:  mov eax, 1
2004:  mov ebx, 0
2008:  cmp ebx, 0
200C:  je 2034
;Basic Block 2
2010:  mov ebx, 2
2020:  mov edx, 14
2024:  mov ecx, 1000
2028:  mov ebx, 1
202C:  cmp eax, ebx
2030:  je 2000
;Basic Block 3
```

```
2034:  inc eax
2038:  ret
```



Basic blocks are fundamental building blocks for constructing a **Control Flow Graph (CFG)**. A CFG is typically generated for one function. Therefore, the process of constructing a CFG from basic blocks belongs to intra-

procedural analysis. Basic blocks of *a function* and their connections collectively form the CFG of this function.

- **Nodes:** each node is a basic block.
- **Edges:** each edge represents an execution path from one basic block to another basic block through a jump or a branching instruction.

By connecting basic blocks based on control flow, a CFG provides a graphical representation of how control passes through a program.

CFGs and basic blocks help analysts to model the program for:

- Understanding the structure of the program
- Profiling the coverage at runtime
- Perform control flow analysis, e.g., constraint and reachability analysis
- Vulnerability discovery
- Guide static or/and dynamic analysis of binaries.

Where to Find More Information?

For more detailed information on working with basic blocks in Ghidra, you can refer to the following documentation:

BasicBlockModel: docs/GhidraAPI_javadoc/api/ghidra/program/model/block/BasicBlockModel.html

CodeBlock: docs/GhidraAPI_javadoc/api/ghidra/program/model/block/CodeBlock.html

Working with Basic Blocks

To generate basic blocks for a binary, you will need to use the **BasicBlockModel** class. Each basic block is a sequence of consecutively executed instructions.

Important Methods in **BasicBlockModel**: Important Methods in **BasicBlockModel**:

1. **getCodeBlocks(TaskMonitor monitor)**: Get an iterator over the code blocks in the entire program.
2. **getCodeBlocksContaining(Address addr, TaskMonitor monitor)**: Get all the Code Blocks containing the address.
3. **getCodeBlocksContaining(AddressSetView addrSet, TaskMonitor monitor)**: Get an iterator over CodeBlocks which overlap the specified address set.
4. **getFlowType(CodeBlock block)**: Return in general how things flow out of this node. (similar to the reference type).

An example - **enumerating all basic blocks in a binary**

```
# Ghidra Scripting: Basic Blocks
# @category: GhidraScripting
# @author: Junjie Zhang

# To enumerate all basic blocks in a binary.
from ghidra.program.model.block import BasicBlockModel
myBlockModel = BasicBlockModel(currentProgram)
myBasicBlocks = myBlockModel.getCodeBlocks(monitor)
```

```
for i in myBasicBlocks:
    print(i)
```

Another example - **enumerating basic block objects in a Function**

```
# Ghidra Scripting: Basic Blocks
# @category: GhidraScripting
# @author: Junjie Zhang

# To enumerate all basic blocks in the current function.

from ghidra.program.model.block import BasicBlockModel
myBlockModel = BasicBlockModel(currentProgram)
currentFunc = getFunctionContaining(currentAddress)
if currentFunc:
    fbody = currentFunc.getBody()
    myBasicBlocks = myBlockModel.getCodeBlocksContaining(fbody, monitor) #fbody is
an instance of the AddressSetView class
    for i in myBasicBlocks:
        name = i.getName()
        print('Name: {}, Starting Address: {}'.format(name,
i.getFirstStartAddress()))
```

Understanding the **CodeBlock** class

The **CodeBlock** class represents these basic blocks in Ghidra. It provides methods for identifying and interacting with individual basic blocks within a program.

1. **Address[]** `getStartAddresses()`: Get all the entry points to this block.
2. **Address** `getFirstStartAddress()`: Return the first start address of the CodeBlock.
3. **FlowType** `getFlowType()`: Return, in theory, how things flow out of this node.
4. **CodeBlockReferenceIterator** `getSources(TaskMonitor monitor)`: Get an Iterator over the CodeBlocks that flow into this CodeBlock.
5. **CodeBlockReferenceIterator** `getDestinations(TaskMonitor monitor)`: Get an Iterator over the CodeBlocks that are flowed to from this CodeBlock.

Understanding the **CodeBlockReference** Class

A **CodeBlockReference** object represents an edge in the CFG. **Nevertheless, it is worth noting that an incoming reference may represent a **DATA** or **INDIRECTION** reference. In such cases, you may want to exclude them from your analysis by verifying the reference types.**

For a basic block object, its `getSources(TaskMonitor monitor)` method will return Iterator of **CodeBlockReference** over the CodeBlocks that flow into this CodeBlock; its `getDestinations(TaskMonitor monitor)` will return an Iterator of **CodeBlockReference** over the CodeBlocks that are flowed to from this CodeBlock.

The `CodeBlockReference` class represents an edge between a source code block (basic block) and its destination code block (basic block).

- When you use the `getSources(TaskMonitor monitor)` method on a `CodeBlock` object, you receive an iterator of `CodeBlockReference` objects. Each `CodeBlockReference` object provides:
 - `getSourceAddress()`: The address of the first instruction in the source code block.
 - `getDestinationAddress()`: The address of the first instruction in the current code block, which is the destination of the edge.
- Similarly, when you use the `getDestinations(TaskMonitor monitor)` method on a `CodeBlock` object, you receive an iterator of `CodeBlockReference` objects. Each `CodeBlockReference` object provides:
 - `getSourceAddress()`: The address of the first instruction in the current code block, which serves as the source of the edge.
 - `getDestinationAddress()`: The address of the first instruction in the destination code block.
- `getSourceBlock()`: Returns the Source `CodeBlock`.
- `getDestinationBlock()`: Returns the Destination `CodeBlock`.
- The `getFlowType()` method is essential. In a `CodeBlock`, when using `getSources()`, it may return references of type `DATA` or `INDIRECTION`. These references do not represent actual incoming edges. Therefore, it is advisable to verify the type of each reference object and discard the irrelevant ones.

```
# Ghidra Scripting: Basic Blocks
# @category: GhidraScripting
# @author: Junjie Zhang

# For one basic block, enumerate all incoming edges and outgoing edges;
# and for each edge, display the source address and the destination address

from ghidra.program.model.block import BasicBlockModel

myBlockModel = BasicBlockModel(currentProgram)

addr = askAddress("Ghidra Scripting: Basic Blocks", "Give me an address of a basic
block:")

myBasicBlock = myBlockModel.getFirstCodeBlockContaining(addr, monitor)

print("Identified Basic Block: {}".format(myBasicBlock.getName()))

incomingEdges = myBasicBlock.getSources(monitor)
outgoingEdges = myBasicBlock.getDestinations(monitor)

while incomingEdges.hasNext():
    i = incomingEdges.next()
    print("incoming edge: src: {}, dest: {}".format(i.getSourceAddress(),
i.getDestinationAddress()))
```

```
while outgoingEdges.hasNext():
    i = outgoingEdges.next()
    print("outgoing edge: src: {}, dest: {}".format(i.getSourceAddress(),
i.getDestinationAddress()))
```

Application: Reachability Analysis

Given a basic block, identify all basic blocks from which the current basic block is reachable.

```
# Ghidra Scripting: Basic Blocks
# @category: GhidraScripting
# @author: Junjie Zhang

#Given a basic block, identify all basic blocks from which the current basic block
is reachable.

from ghidra.program.model.block import BasicBlockModel

myBlockModel = BasicBlockModel(currentProgram)
addr = askAddress("Ghidra Scripting: Basic Blocks", "Give me an address of a basic
block:")
targetBasicBlock = myBlockModel.getFirstCodeBlockContaining(addr, monitor)

if not targetBasicBlock:
    exit()

processedBasicBlocks = set()
toBeProcessedBasicBlocks = [targetBasicBlock]

while len(toBeProcessedBasicBlocks) > 0:
    one = toBeProcessedBasicBlocks.pop(0) # to dequeue the array
    processedBasicBlocks.add(one)
    incomingEdges = one.getSources(monitor)
    while incomingEdges.hasNext():
        incomingRef = incomingEdges.next()
        # an incoming ref can be DATA or INDIRECTION. So you may want to exclude
them.
        if incomingRef.getFlowType().hasFallthrough() or
incomingRef.getFlowType().isJump():
            srcBasicBlock = incomingRef.getSourceBlock()
            if srcBasicBlock:
                if not (srcBasicBlock in processedBasicBlocks):
                    toBeProcessedBasicBlocks.append(srcBasicBlock)

print("The target basic block {} is reachable from the following basic
blocks:".format(targetBasicBlock.getName()))
print([i.getName() for i in processedBasicBlocks])
```

Ghidra Scripting: Working with P-Codes

By **Dr. Junjie Zhang**

Why P-Codes Matter?

P-code is Ghidra’s intermediate representation (IR) used to abstract (a.k.a "lift") assembly instructions across different CPU architectures. It simplifies analysis and enables automated reverse engineering by explicitly representing instruction semantics.

P-code offers a few unique advantages for reverse engineering and program analysis:

- **Architecture Abstraction:** Provides a uniform representation across different CPU architectures.
- **Explicit Side-Effects:** Clearly represents side-effects of assembly instructions that may not be immediately apparent.
- **Built-In Data Flow:** Used for data-flow tracking, taint analysis, and symbolic execution. (Only supported by Refined p-codes though.)

The following table illustrates how these features facilitate two critical program analysis strategies.

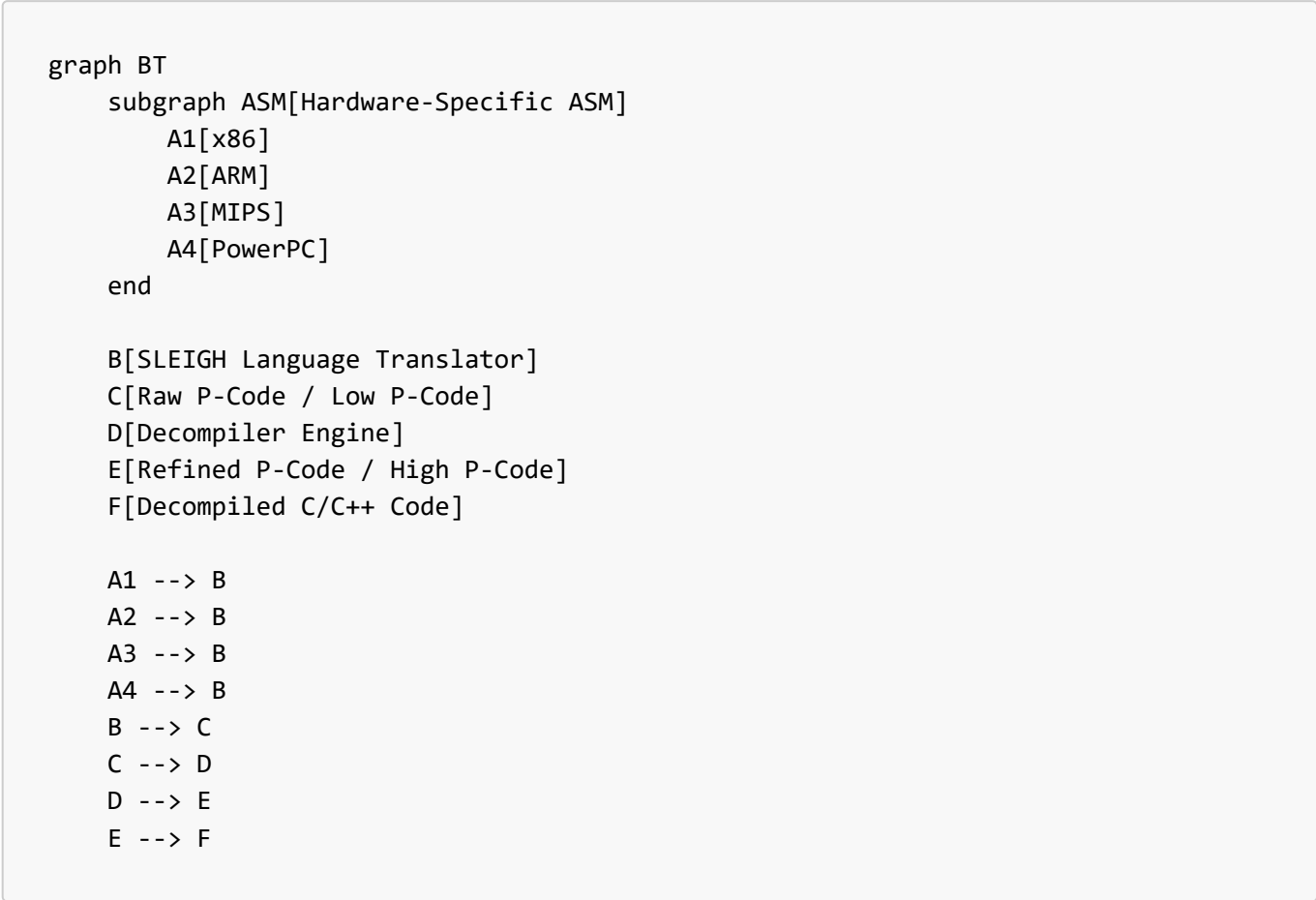
Feature	Emulation	Static Analysis
Architecture Abstraction	X	X
Explicit Side-Effects	X	
Built-In Data Flow		X

Raw and Refined P-Codes

Ghidra defines two types of p-codes that share the same syntax but differ fundamentally in semantics. Although Ghidra itself does not assign specific names to these types, many articles and discussions refer to them as *raw* or *low* p-codes and *refined* or *high* p-codes, respectively.

- **Raw P-code (Low P-code)**
 - **Generated by:** The SLEIGH language translator from processor specification files (**.sla**).
 - **Level:** Low-level, closely mirrors the original machine instructions.
 - **Purpose:** Serves as a detailed, accurate breakdown of the instruction’s hardware-level effects.
 - **Characteristics:**
 - Direct translation from binary opcodes.
 - Represents micro-operations (e.g., shifts, masking, flag updates).
 - Contains temporary registers and precise bit-level manipulations.
 - Not optimized for readability or high-level logic.
- **Refined P-code (High P-code)**
 - **Generated by:** Ghidra’s decompiler after semantic lifting, type recovery, and optimization.
 - **Level:** Higher-level abstraction that aligns with source code constructs.

- **Purpose:** Aims to reconstruct the program's intent and simplify control and data flow.
- **Characteristics:**
 - Abstracts low-level hardware details.
 - Introduces cleaner expressions and temporary variables.
 - In Static Single Assignment (SSA) form
 - Aids in reconstructing functions, loops, and conditionals.



Summary Table - Comparison Between Raw and Refined P-codes

Feature	Raw P-code	Refined P-code
Syntax	P-Code	P-Code
Generated by	SLEIGH	Decompiler engine
Level	Low-level (CPU behavior)	High-level (source logic)
Use case	Emulation, lifting	Decompilation, analysis
Hardware-specific?	No	No
Human readability	Low	High
In SSA form?	No	Yes

Where to Find More Information?

P-Code Overview:

/docs/languages/index.html

Raw P-Code:

/docs/GhidraAPI_javadoc/api/ghidra/program/model/pcode/PcodeOp.html

Refined P-Code:

/docs/GhidraAPI_javadoc/api/ghidra/program/model/pcode/PcodeOpAST.html

P-Code Syntax

A p-code operation is the analog of a machine instruction.

- A Machine Instruction
 - **mnemonic**: the action taken by this instruction (e.g., **ADD**, **JMP**, and etc.).
 - **operand**: the input(s) and output of this instruction (e.g., a register, a memory address, and etc.)
 - **side effects**: a machine instruction may have side effects (e.g., by implicitly changing flag registers.)
- A p-code operation
 - **opcode**: the action taken by this p-code operation
 - **varnode**: the input(s) and output of this p-code operation.
 - **side effects**: for p-code operations, only the output varnode can have its value modified; there are no indirect effects of the operation.

You can easily view raw P-Code instructions directly from Ghidra's GUI. Alternatively, you can extract them programmatically using a simple script, such as the one below.

```
# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

cnt = 0
myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(True)
for inst in instructionIterator:
    cnt += 1
    if cnt > 9:
        break #only display the first few instructions.
    pcodeList = inst.getPcode()
    print("{}".format(inst))
    for pcode in pcodeList:
        print("\t\t{}".format(pcode))
```

Question: Are these p-code instructions raw p-codes or refined p-codes?

Answer: They are raw p-codes directly translated from assembly instructions.

As we can find from the outputs, each of these p-code operations include a p-code opcode/operator, an output if existing, and zero or more inputs.

P-Code Opcode

From </docs/languages/index.html>, you can find the detailed description of various p-code opcodes, which will be essential to the understanding of semantics of p-code instructions.

From </docs/languages/index.html>, you can find the detailed description of various p-code opcodes, which will be essential to the understanding of semantics of p-code instructions.

- COPY
- LOAD
- STORE
- BRANCH
- CBRANCH
- BRANCHIND
- CALL
- CALLIND
- INT_EQUAL
- INT_NOTEQUAL
- INT_LESS: This is an unsigned integer comparison operator.
- INT_SLESS: This is a signed integer comparison operator.
- INT_LESSEQUAL: This is an unsigned integer comparison operator.
- INT_SLESSEQUAL: This is a signed integer comparison operator.

Here are some methods in `PcodeOp` that retrieve information of p-code operators/opcodes.

- `getOpcode()`: get the integer value for this pcode opcode.
- `getMnemonic()`: get the string representation of the pcode opcode.
- `getMnemonic(int op)`: this is a static method to get the string representation of for the integer value of a specific pcode opcode.
- `getOpcode(String s)`: this is a static method get the integer value for a specific pcode opcode in string.

```
# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

cnt = 0
myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(True)
for inst in instructionIterator:

    cnt += 1
    if cnt > 1000:
        break #only display the first few instructions.

    pcodeList = inst.getPcode()
    print("{}".format(inst))
```

```

    for pcode in pcodeList:
        print("\t{}".format(pcode))
        print("\t\topcode in integer and in string: {},
{}".format(pcode.getOpcode(), pcode.getMnemonic()))

print("the opcode string for 1 is {}".format(pcode.getMnemonic(1)))
print("the opcode integer for INT_EQUAL is
{}".format(pcode.getOpcode("INT_EQUAL")))

```

```

# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

#to only print p-code instructions that are `CALL` p-code operators/opcodes

from ghidra.program.model.pcode import PcodeOp
cnt = 0
myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(True)
for inst in instructionIterator:
    pcodeList = inst.getPcode()
    for pcode in pcodeList:
        if pcode.getOpcode() == PcodeOp.CALL:
            print(pcode)

```

P-Code Varnodes

Each input or output is a **varnode**, where **a varnode is a triple of an address space, an offset into the space, and a size. It refers to a contiguous sequence of bytes (defined by the size) starting from the offset in address space.**

Ghidra defines five types of address spaces:

- **ram**: this varnode is in memory. Memory addressable by the program. For example, **BRANCH** and **CBRANCH** can use it, to indicate the address of the execution target.
- **register**: this varnode is in a register.
- **constant**: this varnode is a constant value (i.e., an immediate value).
- **unique** (a.k.a., temporary): this varnode is a temporary node and it does not exist anywhere.
- **stack**: for varnodes in refined p-code instructions.

```

# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

#to explore varnodes

from ghidra.program.model.pcode import PcodeOp
cnt = 0

```

```

myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(True)
for inst in instructionIterator:
    pcodeList = inst.getPcode()
    print("{}".format(inst))
    for pcode in pcodeList:
        print("\t{}".format(pcode))
        output = pcode.getOutput()
        inputs = pcode.getInputs()
        if output:
            print("\t\toutput: {}".format(output))
            print("\t\t\tRam/Address Space?: {}".format(output.isAddress()))
            print("\t\t\tConstant Space?: {}".format(output.isConstant()))
            print("\t\t\tRegister Space?: {}".format(output.isRegister()))
            print("\t\t\tTemporary/Unique Space?: {}".format(output.isUnique()))
            print("\t\t\tOffset: {}, with size: {}".format(output.getOffset(),
output.getSize()))
        for vn in inputs:
            print("\t\tinput: {}".format(vn))
            print("\t\t\tRam/Address Space?: {}".format(vn.isAddress()))
            print("\t\t\tConstant Space?: {}".format(vn.isConstant()))
            print("\t\t\tRegister Space?: {}".format(vn.isRegister()))
            print("\t\t\tTemporary/Unique Space?: {}".format(vn.isUnique()))
            print("\t\t\tOffset: {}, with size: {}".format(vn.getOffset(),
vn.getSize()))

```

Mapping Between Assembly Instructions and P-Code Instructions

From Assembly to *RAW* P-Code

We have explored this in previous sections, i.e., using the `getPcode()` method of the `instruction` class. For example, you can use the following code snippet (incomplete).

```

# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

from ghidra.program.model.pcode import PcodeOp
myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(True)
for inst in instructionIterator:
    #using getPcode(), you will get a list of pcode operations.
    pcodeList = inst.getPcode()
    print("{}".format(inst))
    for pcode in pcodeList:
        print(pcode)

```

From RAW/Refined P-Code to Assembly

Every p-code instruction is associated with the original machine/assembly instruction where it originates from. It should be made clear that one machine/assembly instruction is typically translated into one p-code instruction or a sequence of p-code instruction. Each p-code instruction is uniquely identified by its **sequence number**.

You can use `getSeqnum()` to get an instance of the `sequencenumber` class. And then you can use the following methods:

1. `getTarget()`: returns the address of the assembly instruction from which this P-Code instruction is coming.
 - A sequence number contains **the address** of the original assembly instruction the pcode instruction originates from.
2. `getTime()`: returns the counter value (i.e., the index) of this P-Code instruction among all instructions translated from the assembly instruction.
 - A sequence number also contains a counter. For a single instruction, a **1-up counter**, starting at zero, is used to enumerate the multiple p-code operations involved in its translation.

```
# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

from ghidra.program.model.pcode import PcodeOp
myListing = currentProgram.getListing()
instructionIterator = myListing.getInstructions(True)
for inst in instructionIterator:
    pcodeList = inst.getPcode()
    print("{} : {}".format(inst.getAddress(), inst))
    for pcode in pcodeList:
        print("\t{}".format(pcode))
        seq = pcode.getSeqnum()
        print("\t\tseq number: {}".format(seq))
        print("\t\t\taddress of the assembly instruction this pcode is from: {}
with the index of {}".format(seq.getTarget(), seq.getTime()))
```

Refined P-Code

For all examples we have discussed before this section, they are for **raw P-Code** or **low P-Code**. Here we will focus on refined **P-Code**.

SSA

Refined p-codes are represented in **Static Single Assignment (SSA)** form; raw p-codes are NOT represented in SSA. A key property of SSA is that it will ensure **every variable is assigned exactly once**.

- Introduces new unique versions (e.g., `tmp1`, `tmp2`, etc.) for each assignment.
- Makes **data flow analysis**, **value tracking**, and **optimization** easier.

```
# Yes or No?
# No, because x has been written for 3 times.
x = 5
x = x + 2
x = x * 3
return x
```

```
# Yes or No?
# Yes, since no variable is written for more than once.
x1 = 5
x2 = x1 + 2
x3 = x2 * 3
return x3
```

```
# Yes or No?
# No, since x has been written twice.
if (cond):
    x = 1
else:
    x = 2
y = x + 3
```

```
# Yes or No?
# Yes, since no variable is written for more than once.
if (cond):
    x1 = 1
else:
    x2 = 2
x3 = phi(x1, x2)
y1 = x3 + 3
```

Retrieving Refined P-Code Using Ghidra

You will need to firstly decompile a function before you can retrieve refined p-code instructions from the binary. This is fundamentally different from raw p-code instructions, which can be directly translated from assembly instructions.

```
# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

from ghidra.app.decompiler import *
```

```

# We will get refined p-code for the current function.
func = getFunctionContaining(currentAddress)
myDecomp = DecompInterface()
myDecomp.openProgram(currentProgram)

if func is None:
    print("No function contains this address.")
    exit()

decomp_results = myDecomp.decompileFunction(func, 30, monitor)
if decomp_results is None:
    print("Fail to decompile this function.")
    exit()

results_highFunction = decomp_results.getHighFunction()
if results_highFunction is None:
    print("Fail to get the high function.")
    exit()

pcode_seq = results_highFunction.getPcodeOps() # This pcode_seq is refined pcode!
for op in pcode_seq:
    print("{}".format(op.toString()))

```

Refined P-code instructions are obtained after decompilation. Each varnode in refined p-code instructions can be associated with a variable in the decompiled source code (e.g., in c/c++). The following code shows you how to retrieve the source-level information of a varnode in a refined p-code instruction. Here we print out the variable names in decompiled code for varnodes used as function parameters.

```

# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

from ghidra.app.decompiler import *

paramSet = set()

# We will get refined p-code for the current function.
func = getFunctionContaining(currentAddress)
myDecomp = DecompInterface()
myDecomp.openProgram(currentProgram)

if func is None:
    print("No function contains this address.")
    exit()

decomp_results = myDecomp.decompileFunction(func, 30, monitor)
if decomp_results is None:
    print("Fail to decompile this function.")
    exit()

results_highFunction = decomp_results.getHighFunction()

```



```

if results_highFunction is None:
    print("Fail to get the high function.")
    exit()

pcode_seq = results_highFunction.getPcodeOps() # This pcode_seq is refined pcode!

for op in pcode_seq:
    output = op.getOutput()
    inputs = op.getInputs()
    for invar in inputs:
        hv = invar.getHigh()
        if hv:
            hs = hv.getSymbol()
            if hs and hs.isParameter():
                paramSet.add(invar)

print("all varnodes in refined pcode that are identified as parameters.")
for invar in paramSet:
    hv = invar.getHigh()
    if hv:
        hs = hv.getSymbol()
        if hs:
            print("varnode: {}, high variable name: {} with type {}, symbol name: {}".format(invar, hv.getName(), hv.getDataType(), hs.getName()))

```

getDef() and getDescendants()

Both methods of `varnode` are essential to perform taint or data flow analysis on refined p-code.

- `getDef()`: get the refined p-code instruction that defines this varnode (a.k.a, uses this varnode the output).
- `getDescendants()`: get an iterator of all refined p-code instructions that use this varnode as input.

```

# this code snippet is in the SSA form.
x1 = 1
x2 = 3
y1 = x1 + 5
z1 = y1 * 2
w1 = x1 + x2

```

If you are exploring this instruction `w1 = x1 + x2`, and you get the `x1` variable/varnode in this instruction.

- if you call `x1.getDef()`, you should get the definition of `x1`, i.e., the instruction that writes into `x1`. In other words, `x1.getDef()` should return `x1 = 1`.
- if you call `x1.getDescendants()`, you should get a list of all instructions that use `x1` as the input. In other words, `x1.getDescendants()` should return `y1 = x1 + 5` and `w1 = x1 + x2`, where both of them use `x1` as input.

It helps tracking how data flows in a program by iteratively exploring the definition or descendants of a varnode. For every varnode in each refined p-code instruction, we display its definition instruction and all instructions that use it as input.

```
# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

from ghidra.app.decompiler import *

# We will get refined p-code for the current function.
func = getFunctionContaining(currentAddress)
myDecomp = DecompInterface()
myDecomp.openProgram(currentProgram)

if func is None:
    print("No function contains this address.")
    exit()
decomp_results = myDecomp.decompileFunction(func, 30, monitor)
if decomp_results is None:
    print("Fail to decompile this function.")
    exit()
results_highFunction = decomp_results.getHighFunction()
if results_highFunction is None:
    print("Fail to get the high function.")
    exit()

paramSet = set()
pcode_seq = results_highFunction.getPcodeOps() # This pcode_seq is refined pcode!

for op in pcode_seq:
    print(op)
    output = op.getOutput()
    if output:
        print("\toutput varnode: {}".format(output))

        inst_def = output.getDef()
        print("\t\t It is defined by: {}".format(inst_def))

        inst_descendants = output.getDescendants()
        for i in inst_descendants:
            print("\t\t It is used by: {}".format(i))
    inputs = op.getInputs()
    for invar in inputs:
        print("\tinput varnode: {}".format(invar))
        inst_def = invar.getDef()
        print("\t\t It is defined by: {}".format(inst_def))

        inst_descendants = invar.getDescendants()
        for i in inst_descendants:
            print("\t\t It is used by: {}".format(i))
```

Applications

DecompilerUtils Slicing Methods

- `getBackwardSlice(Varnode Seed)`
- `getForwardSlice(Varnode Seed)`
- `getBackwardSliceToPCodeOps(Varnode Seed)`
- `getForwardSliceToPCodeOps(Varnode Seed)`

```
# Ghidra Scripting: P-Code
# @category: GhidraScripting
# @author: Junjie Zhang

from ghidra.app.decompiler.component import DecompilerUtils
myDecompUtil = DecompilerUtils
```

The Data Flow Visualization in Ghidra